

CoPy: Declarative Object-Oriented Programming Through Copatterns

ADRIANO CORBELINO II and PAUL DOWNEN, University of Massachusetts Lowell, USA

Declarative programming and equational reasoning are more often associated with the functional paradigm than the object-oriented paradigm. However, these concepts can be achieved in object-oriented languages using copattern matching and its dual, pattern matching. While pattern matching allows us to deconstruct data and branch according to its shape, copattern matching allows us to define the behavior of potentially infinite objects via a finite collection of observations. These observations are schematics of a desired matching context that can be replaced by their outcome expression. Copatterns can be nested and mixed with patterns, providing a rich, declarative way to refine the specifications of our context. This work extends Python to support a declarative style of defining objects through copatterns and presents a new encoding of copatterns in terms of classes and method overloading. Our extension is distributed as a Python package, and we define two terse class-based languages named Featherweight Python and Featherweight CoPy. The former is a tiny class-based calculus that captures the essence of Python, and the latter is an extension of it with copattern-based declarations. To nail down this view, we present a series of examples to highlight the expressiveness and compositional capabilities of the system, and prove operational correspondence of our encoding with respect to our theoretical model through a forward simulation.

CCS Concepts: • **Theory of computation** → **Object oriented constructs**; **Operational semantics**; *Equational logic and rewriting*; *Abstraction*.

Additional Key Words and Phrases: Copatterns, Python, Codata, Equational Reasoning, Declarative Programming

1 Introduction

Historically, functional programming (FP) has often been seen as at odds with object-oriented programming (OOP). While OOP serves as the lingua franca in industry, in academic circles that emphasize semantics and correctness, FP is typically promoted and praised for its advantages in (semi-)formal methods, using declarative programming and equational reasoning [11, 17, 10]. Declarative programming lets us focus on what things are instead of explaining how to compute them, and equational reasoning lets us establish properties of programs using familiar mathematical and algebraic methods. The crucial advantage, then, is when the same technique for specifying and reasoning about program behavior can be used as the definition for the program itself. For instance, we can define functions in Haskell by specifying the outcome in terms of the shape of its inputs and follow the principle of “substitution of equals for equals” [17] to reason about the execution of the program. We often see elegant snippets, like the lazy Fibonacci stream (`fibs = 0 : 1 : zipWith (+) fibs (tail fibs)`), being used as paragons for these techniques when promoting functional programming. Implicitly, these examples are usually presented as things only possible in FP, using tools that OOP lacks. But this need not be so!

This work presents an extension to the Python programming language that adds a new, declarative way of defining classes and objects. Similarly to declarative programming in FP, we can declare objects and their behaviors through a list of possible *equations* between (chains of) method calls and their results. Whereas functional languages only permit matching arguments against patterns in these defining equations, our notion of declarative OOP matches against argument patterns *and* the name of methods called at the same time when deciding an answer. In other words, these equations let the OOP programmer use *copattern matching* for defining the behavior of an object depending

```

dec Stack:
  self().push(x)           = self.pushes([x])
  self().pushes(xs).push(x) = self.pushes([x] + xs)
  self().pushes(xs).pop()   = (xs[0], self.pushes(xs[1:]))

```

Fig. 1. A stack object defined through copatterns in CoPy.

on the context it is used in. Moreover, this same notion of copattern-matching objects gives a model for equational reasoning with object-oriented programs. Declarative OOP provides a powerful tool that enables us not just to answer questions of the form: "What is the result of `reverse` when we provide the empty list?" (`reverse [] = ...`) but also questions like "What should be the result of `open("file.txt")` followed by `read()` (`open("file.txt").read() = ...`)."

To demonstrate this approach, consider the example of a stack object defined through copatterns on fig. 1. In this example, we use three equations to define how the object should respond to the messages `push` and `pop`. These equations are enough to specify behaviors like failing when popping an empty stack and returning the last pushed element when popping a non-empty stack. The intricacies of this style will be developed in the following sections, but the main point is that we can define an object's behavior through a collection of equations and then use those equations to reason about the program's behavior.

Even with the observational nature of copatterns, they are most often present within the functional realm. Copatterns have their most mature implementation in Agda [3], and are also present in Beluga [14, 15], Ocaml [13], and Racket [6]. In this scene, we bring a declarative style of object-oriented programming via copattern matching to a mainstream OOP language. We chose Python as our target because of its dynamically typed nature and its sufficient, though somewhat hidden, extensibility. In addition, we believe that bringing copatterns to a mainstream language in an accessible way is a crucial part of its maturation process, since it can increase their exposure to realistic programming tasks. Our extension defines a new statement to allow a declarative style of object definition, and is distributed as a portable Python package, following the spirit of [16], thereby easing access to our work. Specifically, our main contributions are:

- **A conservative Python language extension for declarative copattern-based object definition.** Section 2 introduces our extension functionality and brings the spotlight to what we can do with copatterns through a series of examples. Section 3 presents the definition of our extension's API, and section 6 discusses the implementation details, technical challenges, and limitations.
- **A family of languages to model copattern-based definitions and their translation to regular classes** In section 4, we define two nested language and their single-step operational semantics in order to model the semantics of copattern-based definitions and their translation to regular classes. The source language, Featherweight CoPy (Fw_{py}^{co}), is a core calculus for an OO copattern-based class declaration, and captures the essential features of CoPy. The target language, Featherweight Python (Fw_{py}), is a terse class-based language inspired by Python, following a similar approach to Featherweight Java [12].
- **Translations showing how to implement copattern-based definitions in terms of classes and how to implement classes with dictionaries.** In section 5, we present the translation $\llbracket _ \rrbracket$. This translation explains how to implement (co)-pattern matching using regular classes and captures the technique used in our implementation. To assert its preserve meaning, we have shown its soundness via a forward simulation between the operational semantics of the languages.

2 A Declarative Approach to Object-Oriented Programming

The contemporary public perception of Object-Oriented Programming (OOP) is often tied to the ideas of classes, inheritance, mutable state, and its use in the procedural world. The literature also presents theoretical models for the paradigm-defining features of OOP. For example, the work of Cardelli [1] focus on the definition of OOP constructs in terms of the λ -calculus. This result enable us to use functional techniques, like defining and reasoning about programs in terms of equations, after we have gone through Cardelli's encoding. In contrast, this work presents a way to use these same functional techniques directly in OOP, without an indirect translation step, by introducing a new flavor of OOP: one that is declarative, compositional, and supports native equational reasoning. This section presents a series of examples to illustrate how we can achieve this goal using our main construct, copatterns, and reason about OOP programs in a methodical way.¹

2.1 Why Copatterns: The relation between Objects and Codata types

As previously stated, we use copatterns as the main character of this new style of OOP. But the curious reader might wonder: why copatterns and why put a feature associated with FP into OOP? The answer is that copatterns, and their related operation copattern matching, are a natural way to define and work with abstract objects—encapsulated, self-aware, dynamically-dispatching black-boxes that we can only interact with through a pre-defined interface of operations—in the form of codata types. While codata types have a long history in the theory and foundations of computation, they are not as popular as inductive data types for everyday programming. These types show their brightest colors when we want to define objects by describing their behavior instead of their structure. To illustrate this point, usually, we define functions by describing their behavior; in functional languages, we often use patterns to define how the function should behave for different inputs. This is not limited to the functional world: we can extend this line of thought to the procedural world where a function is defined as a sequence of steps whose internals lack a concrete, scrutable representation at run-time. This same approach can be applied to OOP, where we can define objects *only* by describing their behavior when responding to messages (*i.e.*, methods).

2.2 CoPy: How to represent classes

<pre>class Square: def __init__(self, s): self.side = s def area(self): return self.side * self.side def perimeter(self): return self.side * 4</pre>	<pre>dec Square: self(s).area() = s * s self(s).perimeter() = s * 4</pre>	<pre>class Square_handlers: ... class Square(CoData): ... __metadata__ =</pre>
(a) Regular Python class.	(b) CoPy class.	(c) Generated class.

Fig. 2. Different ways of defining a class.

In order to introduce our language extension, let us start with one of the most fundamental building blocks of OOP in Python: classes. The entry point of our extension is the `dec` statement. This statement let us to `declare` a class by describing a series of copattern matching equations that

¹Most snippets in this section are written using our language extension, which requires a brief setup process explained in detail on Section 3.

we called observations. To make things concrete, let us begin with a simple example class, **Square**, whose objects know how to respond to two methods: **area** and **perimeter**. In regular Python, and in many standard OO languages, we would define this class as shown in Figure 2a, where we define the class by providing its name, and definitions for its constructor (`__init__`) and methods (**area** and **perimeter**) as procedural statements.

Note that here we are describing how each instance of **Square** should know its given size and should respond to calls of **area** and **perimeter**. Alternatively, we can take a more declarative approach by utilizing the `dec` statement, as shown in Figure 2b. Similarly, this statement expects a class name, but instead of providing implementations for the constructor and methods, we provide a (*order-sensitive*) sequence of observations.

Each observation describes a specific interaction with the class, its constructor, and the objects it builds, by specifying the blueprint describing a use-case as a context, and what should happen when we find a compatible context. This blueprint, called a *copattern*, is a schematic of a desired matching context that will be substituted by its outcome during a successful copattern match. In the case of **Square**, we have two observations for calling a method—one for **area** and one for **perimeter**—*immediately* after an object is constructed. The first component of a copattern always denotes the construction of an object; in **Square**'s observations, the `self(s)` stands for a constructor case matching **Square**(*n*) where *s* gets bound to *n* and `self` is bound to the new **Square** object returned by the constructor. From there, the first observation states that if we have an object of the form `self(s)`—read as a new object (`self`) who had exactly one constructor argument (*s*)—followed by a call to the **area** method, it should be replaced with $s * s$. Likewise, the second observation specifies how this class of object should answer to the **perimeter** method, by replacing `Square(s).perimeter()` with $s * 4$. The important remark here is the structure of the copattern in each observation; in those copatterns, we describe contexts that put strict requirements on how the object was created (the number of constructor arguments, and possibly their shapes and resulting bindings) and how they should respond to certain interactions with their attributes and methods.

Our implementation of CoPy translates these declarations into plain Python classes. We can see the simplified sketch of how the **Square** declaration is translated in Figure 2c. Here, it suffices to notice that each declaration is transformed into a class inheriting from **Codata**—a common base class giving objects the ability to perform copattern matching—and carrying a list of observations as a class attribute. The full details of this translation are presented in the implementation described later in Section 6. For now, we will illustrate what this means for high-level programming and how we can use and reason about it with a series of examples.

Having definitions is not the full story, after all; it is not enthralling if we cannot use them. Thus, we shall see how we can create instances and interact with them. In a regular Python class, we create an instance by calling the class name with the arguments specified by the constructor — `Square(7)` —, and we interact with it by calling its methods and accessing its attributes — `Square(8).side` and `Square(9).area()` —. Happily, we do not need to learn new syntax to perform the same operations with our new declarations. We can create an instance of **Square** by writing `Square(7)`, and we can interact with it by calling its methods and accessing its attributes, for example, `Square(8).side` and `Square(9).area()`, using exactly the same syntax! Under the hood, when we create an instance of **Square** by calling `Square(7)`, we are creating an object of the generated class (the one inheriting from **Codata**) with the given arguments, and when we interact with it, we are performing a copattern match on the object to find a compatible observation and replace the context with the outcome of the first matching observation. But some cans of worms are better appreciated later; thus, we will see the details of how this process works after we better understand the capabilities of our extension.

2.3 Copatterns 101: Mixing & Nesting

Up to now, we have only seen simple copatterns mimicking regular classes, attributes, and methods. These do not do justice to the full potential of copatterns, which can be more elaborate and powerful. Let us scale up this style by building a representation of a file system with two types of objects: files and directories. In this example, our file system model will carry the root of the file system, and we will model its entries. To begin, a generic file system entry should remember its name:

```
dec Entry:
    self(n).name = n
```

Here we have a single observation specifying how an object (`self`) created with a single constructor argument (`n`) should respond to a query about the attribute query `name`. Notice the equational nature—even hinted in the syntax—of these copatterns declarations. Consider the evaluation of a basic example: `Entry("secrets.md").name`. The principal way of interacting with declarations is through copattern matching. During this operation, we look for the first suitable observation. Since we have a compatible context—an object constructed with a single argument (`Entry("secrets.md")`) followed by an attribute access (`.name`)—the copattern match will replace the full expression `Entry("secrets.md").name` matching the copattern (*i.e.*, left-hand side with variables bound to arguments) with the outcome (*i.e.*, the right-hand side evaluated with those bindings) of the first compatible observation, returning `"secrets.md"`.

```
> Entry("file.txt").name
"file.txt"
```

If we try to interact in any other way, we will get an `InvalidCoMatch` exception since the copattern match will not find any compatible observation:

```
> Entry("file.txt").path
InvalidCoMatch(...)
```

Now with this intuition, what should be the result of `Entry(5).name`? Could it be 5? Yes, indeed it is, since the copattern match will find the first compatible observation, which is the one we have, and it will replace the context with the outcome of this observation, which is 5. Except this might not be what we expected, since we probably wanted to have the name of an entry to be a string, and not any value. We can straightforwardly fix this by using a popular friend: patterns! By changing the copattern to `self(str(n))`, we are saying that we want to match objects created with a single constructor argument, but we also want this argument to be a string, and we want to bind this string to `n` for use in the outcome of the observation. Now we gracefully fail if we try to create an `Entry` with a non-string value:

```
> Entry(True).name
InvalidCoMatch(...)
```

This highlights one of the main features of copatterns: they can contain patterns, thus increasing the granularity of these contexts. We are not limited to using shallow patterns in the constructor arguments, they can be nested, and we can also use them in call copatterns, like `self.double(int(n))`. In summary, we support a variety of patterns:²

- Variable patterns (`x`), which bind any value to a variable.
- Literal patterns (`True`, `None`, `42`, `[]`, `"115"`), matching values against literals.
- Primitive type patterns (`int(n)`, `str(s)`, `float(f)`, `bool(b)`), which require the matched value to be of a given primitive type.
- List patterns (`[x, *xs]`), which require the matched value to be a non-empty list.

²Even though recent versions of Python support pattern matching, we have implemented our own version for the sake of extensibility.

- Container patterns (`[x, y, z]`, `(int(n), bool(b))`, `{"e": str(_)}`), which require the matched value to be a container with the given elements.
- Class patterns (`MyClass(x, y)`), which require the matched value to be an instance of a given class. Class patterns also bind and expose the variables in `__match_args__`.

However, there is nothing too exciting about having an object with one attribute, so consider how to define a **File**. A file should know its name and its content.

```
dec File(Entry):
    self(name, str(ctt)).ctt = ctt
    self(_, str(ctt)).size = len(ctt)
```

This illustrates two new features. Right on the first line, we can see that CoPy supports inheritance with the same syntax used in regular classes. This inheritance allows us to compose declarations and permits us to easily create partial definitions and group them. At the moment, it is enough to imagine that we append the observations of the inherited classes. Lastly, in the second observation, we can see an instance of the wildcard pattern (`_`). This is a pattern that is always valid and does not create a binding. We use it to create the constraint that this object is supposed to have two constructor arguments, but we are flagging that we will not use the first value. Now, we can create files that know how to calculate its size and return its contents. To exemplify:

```
> File("f.txt", "tiny").size
4
```

Nonetheless, this definition probably does not meet our expectations. Consider what should happen when we ask the name of a file (`File("f.txt", "tiny").name`). It should be `"f.txt"`, right? After all, we inherit from **Entry** which defines this behavior. Sadly, if we execute this example, we learn that we do not have a compatible observation.

```
> File("f.txt", "tiny").name
InvalidCoMatch(...)
```

This might be shocking, one may inquire: how about the **Entry**'s observation? We are inheriting this interaction. Why is this not compatible? Is there a bug in the implementation? In reality, no, this is the desired semantics—after all, a method call with too many arguments should report an arity-mismatch error, instead of silently truncating the extras—and gives us the opportunity to introduce one more facet of copatterns: *self copatterns*! Notice how this interaction is not compatible with the given observation (`self(str(n)).name = n`) because it handles the case when the constructor is given exactly one argument. Yet here we were creating Files with two constructor arguments, so the comatch fails at the first step. Okay, but how can this obvious limitation be fixed? Should we add a new observation, `self(str(n), _).name = n`, to **Entry** for handling the new case? This is a valid solution, but things pile up pretty quickly if we follow this preemptive approach. Being able to freeze a specific number and structure of arguments is useful, but would it be nice if we had a more flexible way of creating constraints on instances?

The answer to this conundrum is the self copattern. Instead of using parentheses around the constructor arguments, mimicking the constructor syntax, we can use curly braces to switch to a different kind of self copattern. For example, `self{str(n)}.name = n` matches a context where an object is constructed with *at least* a string as its first constructor argument. The important difference here is that the object is allowed to have *more* attributes, and may or may not say anything regarding the order of the constructor arguments. In this specific example, we are matching against the order of the constructor arguments, but we could also match against the name of the attributes (*order insensitive*), using the syntax `self{attr: pattern}`. This kind of self copattern is akin to row polymorphism [18], where records are permitted to hold more information than the ones specified

```

1  dec Entry:
2      self{str(n)}.name = n
3
4  dec File(Entry):
5      self{_, str(ctt)}.ctt = ctt
6      self{_, str(ctt)}.size = len(ctt)
7
8  dec Dir(Entry):
9      self{_, es: list(entries)}.size = sum(map(lambda e: e.size, entries)) + 8

```

Fig. 3. Final definitions for the file system example.

```

1  dec FP:
2      __.open(str(p)) = FP(True, p)
3      self{False, p}.open() = FP(True, p)
4      self{True, p}.read() = open(p).read()
5      self{True, p}.close() = FP(False, p)

```

Fig. 4. Definition of a file pointer class.

in the pattern, but they must at least hold compatible values for the ones specified in the pattern. Substituting the original copattern with a self copattern, we can now have the expected behavior:

```

> File("f.txt", "tiny").name
"tiny"

```

We can finish this example by defining **Directory**. This declaration states that a directory should know its children and how to calculate its size. The size of a directory is the sum of the sizes of its entries plus 8 (the size of the directory itself). The full declaration is shown in fig. 3. To assert our understanding of the complete version, consider the following values for the next execution example: a file `f = File("copy.pdf", "top secret")`, and two directories, `d = Dir("src", es=[])` and `conf = Dir("papers", es: [f, d])`

```

conf.size = sum(map(lambda e: e.size, [f, d])) + 8           (line 9)
            = f.size + d.size + 8                           (sum & map)
            = File("`copy.pdf'", "`top secret'").size + d.size + 8   (f)
            = len("`top secret'") + d.size + 8                   (line 6)
            = 10 + Dir("`src'", "`es'=[]).size + 8              (len & d)
            = 10 + sum(map(lambda e: e.size, [])) + 8           (line 9)
            = 10 + 8 + 8                                         (sum & map)
            = 26                                                 (+)

```

Keeping the spirit of the previous example, consider this next class representing a file pointer. An instance of this declaration represents a pointer to a file and keeps track of its state: open and closed.

The first observation states that if we have a call to the method `open` with a string argument, we should create a new file pointer with the state open and the given path. The following three observations describe how instances of **FilePointer** should respond to the `open`, `read`, and `close` methods, depending on their state (open or closed). Let's test this definition by seeing what happens

if we try to close an already-closed file pointer:

```

FP.open("a").close().close() = FP(True, "a").close().close()      (line 2)
                             = FP(False, "a").close()             (line 5)
                             = InvalidCoMatch(...)                (comatch failure)

```

OOPS, it seems we do not have a compatible observation for the last call to `close`, since we do not have an observation describing how a closed file pointer should respond to a call to `close`. We can easily fix this by adding the following observation: `self{False, p}.close() = self`, but this is a good segue for discussing copattern coverage. Being too specific in our observations can lead to copattern match failures, if we forget to cover some cases, which can be a good thing if we want to be strict about the protocols (e.g., only open files can be closed) users of the class must follow. However, this extra flexibility can be undesirable if we judge some interactions to be irrelevant for the behavior of our objects, and we want to avoid the overhead of writing additional observations for these irrelevant interactions.

Above all, we are in a fully-fledged programming language where partial functions are mundane, so partial definitions are already possible—and even common—in our language, but we should keep an eye on the consequences of this choice. Due to Python’s dynamic nature, the presented version of CoPy does not support a way to check the coverage of our observations, but we plan to add coverage checking in the future (section 8) to automatically reason about the completeness of our definitions. For now, users should be careful about the coverage of their observations, and they should be aware of the consequences of partial definitions.

The icing on the cake, which makes copatterns distinctly more appealing than implementing each method separately, is that copatterns can be nested into chains of multiple method calls. This means that we can outline contexts with arbitrary depth, and put constraints on them with patterns, going beyond simple contexts analogous to regular attribute and method definitions. For example, nested copatterns are useful when we want to define a class `Player` (fig. 5) that represents a player in a game. A player can take multiple actions, but we will focus on two of them: `guard` and `take_damage`. Our objective here is to implement a simple combat system, where a player can guard against an attack, and if the player is attacked while guarding, the attack is avoided, but if the player is attacked without guarding, the player receives damage. This declaration might seem a bit

```

1  dec Player:
2      ...
3      self{}.guard(int(d)).take_damage(int(a)) = a - d
4      self{}.guard(int(d))                    = 0
5      self{}.take_damage(int(a))               = a
6      ...

```

Fig. 5. Definition of a player class using nested copatterns.

off at first, but let us dissect it with a couple of examples. Here we are using an integer to calculate how much damage the player received. The last two observations are straightforward, working as regular methods. However, the first observation is a bit more interesting; one could even argue it is dislocated, since one could prefer to describe the simple actions before the combined actions. This observation states that if we have a call to `take_damage` preceded by a call to `guard`, we should return less damage than an unguarded attack. Noticing this strangeness, let’s use our familiar and

friendly equations to see how this works!

```
Player().guard(3).take_damage(5) = 0.take_damage(5)      (line 4)
                                = AttributeError(...)
```

Right? But actually, no, the previous example is improper since we did not respect the order of the observations. Paying close attention to the observations, we can see that the first observation is, in fact, the first compatible observation. This might look too magical, especially if we consider actions like, where `p` is `Player().guard(3)`:

```
> p == 0          > p.take_damage(5)      > p.guard(4)
True              2                        AttributeError: ...
```

It seems that `p` returns `0`, but somehow it also has a method `take_damage` which is not defined for booleans. How is this possible and consistent? The answer is that during a copattern match, we might have partial matches. These matches represent a non-definitive result; they signal we have a potential observation, but need more information to make a final decision. To respect the order of observations, we return a resumable copattern match object that can continue matching when additional information becomes available. This is exactly the case of `p`, with only a call to `guard`: we have a partial match with the first observation, but we need to wait for the call to `take_damage` to know whether this is the right observation. If we witness a call to `take_damage`, we can resume the copattern match, and we will find that the first observation is compatible with the new context, thus we return `0`. On the other hand, if we witness an interaction that invalidates the first observations, we can resume the copattern match and keep looking for an observation compatible with part of the context. For example, a call to `guard` on `p` will find that the first observation is not compatible with the new context, thus we will continue the copattern match and find that the second observation is compatible with a part of the context, resulting in `0.guard()`.

We can observe the composite nature of copatterns and their partial matches in the following example, where we create an object that respects the spirit of a fluent API [9]. To illustrate, consider this `Tab` declaration that represents a restaurant tab. Our tab can be open or closed, and it can have a list of items. When we open a tab, we create an empty tab that we can add items to, but we can only request the total price when the tab is closed. Here, the first observation serves as the entry

```
1  dec Tab:
2      _ .open()                = Tab().items([])
3      self().items(xs).add(float(x)) = self().items(xs + [x])
4      self().items(xs).close()    = sum(xs)
```

Fig. 7. Definition of a tab class with a fluent API.

point to our tiny fluent API, which we can access only after completing the first observation. Inside this API, we can add items to our tab, but we can only get the total price when we close the tab. The important remark here is that we are manipulating the context by returning partial matches. To make this explicit, our constructor-esque interaction `open` returns a partial match with the last two observations. The second observation keeps updating this partial match, and using it as a piece of state, until we call `close`, producing the total price.

3 CoPy: Syntax and Programmer API

Now that we become familiar with what copatterns enable us to do, let us define what tools our extension provides. Our primary abstraction is a co-object, an object capable of copattern matching. The objects are defined through `dec` statements (fig. 8).

$\begin{aligned} \text{Dec} &::= \text{dec } Id(\overline{Id}) : \overline{Obs} \\ \text{Obs} &::= \text{Cop} = \text{Out} \mid * Id \\ \text{Cop} &::= \text{Root} \mid \text{Cop}.Id \mid \text{Cop}(\overline{PP}) \mid \text{Cop}\{\overline{PP}\} \\ \text{Root} &::= _ \mid \text{self} \\ PP &::= Pat \mid Id : Pat \\ \text{Out} &::= \text{python_expr} \mid \text{do} : \text{python_block} \end{aligned}$	$\begin{aligned} \text{Pat} &::= Lit \mid \text{Type}(Id) \mid Id \mid _ \\ &\mid \text{Class}(\overline{Pat}) \mid [\overline{Pat}(*Id)^?] \\ &\mid (\overline{Pat}) \mid \{\overline{Pat}\} \mid \{\overline{Lit} : \overline{Pat}\} \\ &\mid (Pat \mid Pat) \mid Pat \text{ as } Id \\ \text{Lit} &::= \text{True} \mid \text{False} \mid \text{None} \\ &\mid \text{"str"} \mid b\text{"bytes"} \\ &\mid \text{python_number} \\ \text{Type} &::= \text{int} \mid \text{float} \mid \text{str} \mid \text{bool} \\ &\mid \text{list} \mid \text{dict} \mid \text{set} \mid \text{tuple} \\ &\mid \text{bytes} \mid \text{complex} \end{aligned}$
---	--

Fig. 8. Syntax of a `dec` statement.

Declarations & Observations. A `dec` statement specifies which observations should be in a given co-object. The signature of this statement is similar to the class statement: we define a name for a new class, its superclasses, and, as mentioned in the previous section, a sequence of observations (*order sensitive*). Each observation contains a copattern and an outcome.

Outcomes. The outcome might be a simple Python expression, like the body of a `lambda`, or if we start with `do`: `...`, a full-fledged block. We borrow Haskell’s `do` syntax to stress the imperative nature of a code block. Regardless of the outcome type, they will be able to access the bindings defined in the copattern on a copattern match success.

Copatterns. The structure of a copattern resembles a series of attributes and applications. This sequence starts with a root followed by a series of copatterns fragments, named experiments, which refine the desired matching context. The root can be an `_`, saying that we don’t require anything specific to the instance, or a `self`, which may be followed by one fragment to be used with the instance construction. If we have a $(\overline{PP})$, we require the object to have exactly the structure defined by those $\overline{PP}$. But, if we have $\{\overline{PP}\}$, we change to a row-polymorphism-like semantics telling that the instance should satisfy at least those constraints, but it is allowed to contain more information. These row parameter patterns are only permitted to be used just after the root. To materialize, think that these copatterns are being used against the constructor call. A parameter pattern (PP) can represent a positional argument (Pat) or a named parameter ($Id : Pat$).

Patterns. We have implemented our own pattern-matching system. We have two main reasons to pursue that route. First, we want control over what, when, and how the process works. Python provides limited support for extensibility of the pattern match statement, mainly done via `__match_args__`. This customizability will be useful for future work for features such as interfaces and coverage checking. Second, we want to be compatible with older versions of Python, and Python’s structural pattern match statement is fairly new (Python 3.10). Our system aims to be compatible with what users already expect from Python’s `match` statement (PEP 634), but we do not aim to be fully compliant. Our system supports the following patterns:

- A literal pattern, specifying a specific value.
- a type pattern, performing a check against a value type.
- A variable pattern, not imposing any specifications.

- A wildcard pattern, which is similar to a variable pattern, but does not generate any binding.
- A Class pattern, with a semantic consistent with `isinstance` and using `__match_args__` to check the attributes.
- A list pattern, allowing an arbitrary prefix of elements followed by an optional binding for the tail of the list.
- Container patterns, permitting strict requirements upon the shape and number of elements in containers;
- Or patterns to provide alternatives when the first pattern fails.
- As patterns to provide a different bind, similar to the use of `@` in Haskell.

Inheritance & Composition. Declarations can inherit from other classes. These superclasses can be both declarations and regular classes. When inheriting from a regular class, the expected behavior is to make the super methods and attributes available and follow the method resolution order (MRO). Members inherited from a Python class have precedence over starting a copattern match. For example, `dec Nats(StreamMethods): ...` defines a declaration named `Nats` that inherits from a regular class `StreamMethods`. If `StreamMethods` defines a method named `take`, the interaction `Nats().take` will return a reference for the method instead of starting a copattern match. Inheriting from a declaration is a form of composition. The super's observations become available, which can be interpreted as appending them to the current observation list. However, we have two other ways of composition. We can use splat syntax (`*Id`) in the place of an observation to specify the exact spot where the observations of `Id` should be inserted, permitting more fine-grained control over how to merge observations list. Lastly, we can perform dynamic runtime composition using the class method `compose`. This method is available for every declaration, and it returns an object capable of performing the bookkeeping for the observation list and recursive calls— using the whole composition for `self` —.

Installation & Usage. Even though our extension is available as a Python package, we still need to follow a brief setup process to access the new features. Since we add new syntax, we need to use one of our two entry points to teach Python how to parse the new constructs. The first one is an import hook. Import hooks enable us to define custom behavior when importing modules. After installing the package, to use the hook, include the following code before importing from `copy` files: `from CoPy.loader import setup_hook; setup_hook()`. The second entry point is a custom text codec. This approach enables us to use the new syntax in a regular `.py` file, if we start the file with the following line: `# -*- coding: copy -*-`. To use this method, we need to register CoPy's codec, which can be done by executing `register_copycodec.py`, provided by the package.

4 Fw_{py}^{co} & Fw_{py} : Models for Copatterns and Classes

To validate our encoding of copatterns and to define the expected semantics more rigorously, we use two languages as our theoretical model. We opt to define two terse languages, alongside their small-step operational semantics, with only the required features to highlight our approach. Providing a complete model for a full-fledged programming language like Python is a daunting task and worthy of its own individual work. Thus, we define these languages to highlight the implementation of copatterns in terms of classes, inheritance, and dictionaries. Featherweight Python (Fw_{py}) is a tiny class-based language including the primitives required for our extension. Featherweight CoPy (Fw_{py}^{co}) is our model for CoPy. It is an extension of Fw_{py} with copattern-based declarations.

$$\begin{aligned}
& \text{Variable} \ni x, y, z \quad \text{Attribute} \ni a, b, m \quad \text{Class} \ni C \\
& \text{Literal} \ni l, n ::= 'a \mid n \mid \text{None} \mid \text{False} \mid \text{True} \\
& \text{Expression} \ni e ::= l \mid x \mid \lambda x. e \mid e_1(e_2) \mid \overline{\{l : V\}} \mid e_1[e_2] \mid e_1 + e_2 : e_3 \mid e_1 ? e_2 \\
& \quad \mid \text{if } e_1 \text{ then } e_2 \text{ else } e_2 \mid \text{fold}(e_1, e_2, e_2) \mid e_1 == e_2 \mid e.m \mid D; e \mid \text{new } x := C(e_1); e_2 \\
& \text{Value} \ni V ::= l \mid x \mid \lambda x. e \mid \overline{\{l : V\}} \mid D; V \\
& \text{Definition} \ni D ::= \text{class } C : I; \overline{M} \mid \text{new } x := C(V) \\
& \text{Constructor} \ni I ::= \text{init}(x) : \overline{a_j = e_j^j} \\
& \text{Method} \ni M ::= m(\text{self}, x) : e \\
& \text{Frame} \ni F ::= \square(e) \mid V(\square) \mid \square[e] \mid V[\square] \mid \square + e_2 : e_3 \mid V_1 + \square : e_3 \mid V_1 + V_2 : \square \\
& \quad \mid \square ? e_2 \mid V_1 ? \square \mid \text{fold}(\square, e_2, e_3) \mid \text{fold}(V_1, \square, e_3) \mid \text{fold}(V_1, V_2, \square) \\
& \quad \mid \text{if } \square \text{ then } e_2 \text{ else } e_3 \mid \square == e_2 \mid V_1 == \square \mid \square.m \mid \text{new } x := C(\square); e \\
& \text{EvalCtx} \ni E ::= \square \mid F[E] \mid D; E
\end{aligned}$$

Fig. 9. Syntax of Fw_{py} — Classes and Objects.

4.1 Fw_{py} : A tiny class based language

Our starting point is Fw_{py} , a language mixing the λ -calculus with classes, objects, and dictionaries. This language has a standard set of literals, variables, λ - abstractions, dictionaries mapping literals to values, dictionary operations, if expressions, and equality between values. Dictionary indexing is done through $e_1[e_2]$, $e_1 + e_2 : e_3$ represents an insertion of a new key-value pair, and we can check if a dictionary contains a key with $e_1 ? e_2$.

Our first step toward copatterns is classes and their related machinery. The term $D; e$, exposes the definition D to e . A definition can be either a class definition or an object instance. Our notion of a class includes a name, a constructor, and a set of methods. One important remark is that both lambdas and methods expect a single argument. We emulate multiple, positional, and named arguments using dictionaries. The constructor allows us to specify how to set up instance attributes based on the constructor argument. Each method has a name, a binding to the instance, and an argument. An object instance assigns a name to an object and records its class and instance attributes. The term $\text{new } x := C(e_1); e_2$ represents a C-object in creation, where e_1 will evaluate to the object instance attribute dictionary. Object member access uses the dot syntax: $e.m$.

Lastly, we take the liberty of splitting the usual definition of evaluation contexts into two syntactic categories: frames and evaluation contexts. Frames guide us to the next step, and evaluation contexts allow arbitrary depth. Since we are not modelling effects, this distinction is useful to model definitions look-up in our operational semantics.

Since this is a terse language, we left some constructs as syntactic sugar; we define them here for clarity. We often use the standard let binding encoding, $\text{let } x = e_1 \text{ in } e_2$, which is sugar for $(\lambda x. e_2) e_1$. Dictionaries with features containing expressions in their keys or values are sugar for regular dictionary values and a sequence of insertions: $\{\overline{fv}, \overline{e_1 : e_2}\} \stackrel{\text{def}}{=} \{\overline{fv}\} + \overline{e_1 : e_2}$. Our model only supports single-argument lambdas and methods. However, since this argument needs to be a dictionary, we can emulate multiple positional and named arguments using a single dictionary. We use indexes to access positional arguments by their position, and x_{args} is the dictionary containing the named arguments — the same one that appears in the call —. An application with multiple arguments is

(ctx)	$E[e] \mapsto E[e']$	(if $e \mapsto e'$)
(β)	$(\lambda x.e)(V) \mapsto e[V/x]$	
(attr)	$\{l : V; \overline{l_i : V_i^i}\}[L] \mapsto V$	
(ins)	$\{\overline{l_i : V_i^i}\} + l : V \mapsto \{l : V; \overline{l_i : V_i^i}\}$	
(if _t)	if True then e_1 else $e_2 \mapsto e_1$	
(if _f)	if False then e_1 else $e_2 \mapsto e_2$	
(eq _t)	$l == l \mapsto \text{True}$	
(eq _f)	$l_1 == l_2 \mapsto \text{False}$	(where $l_1 \neq l_2$)
(in _t)	$\{l : V, \overline{l_i : V_i^i}\} ? l \mapsto \text{True}$	
(in _f)	$\{\overline{l_i : V_i^i}\} ? l \mapsto \text{False}$	(where $l \notin \overline{l_i^i}$)
(fold ₁)	fold ($V_f, \{\}, V_0$) $\mapsto V_0$	
(fold ₂)	fold ($V_f, \{l : V, \overline{l_i : V_i^i}\}, V_0$) $\mapsto \text{fold}(V_f, \{\overline{l_i : V_i^i}\}, V_f(V_0, l))$	

Fig. 10. Operational semantics the other constructs of Fw_{py} .

sugar for an application with a single dictionary argument: $e_1(\overline{e_i^i}, \overline{x_j = e_j^j}) \stackrel{\text{def}}{=} e_1(\{i : \overline{e_i^i}, \overline{x_j : e_j^j}\})$. We encode lists as dictionaries with consecutive integer keys starting from zero: $[\overline{e_i^i}] \stackrel{\text{def}}{=} \{i : \overline{e_i^i}\}$. We model single inheritance by appending the methods of the parent class into the child class.

The operational semantics of Fw_{py} is given on figs. 10 and 11. The reduction rules in fig. 10 are mostly standard, while the ones in fig. 11 are specific to our class-based constructs. For the class constructs, we define their semantics in terms of their evaluation contexts and frames.

- (*decl*) allows us to extract a definition to push a frame, if the frame does not contain the variable bound by the definition.
- (*new*) specifies how to create a new object instance, by inserting a new definition. It looks up the closest binding for the class C , and it uses its constructor to produce a term that represents an object in creation.
- (*attr*) guides the semantics of an instance attribute access. We look for the attribute dictionary of the closest binding for our instance.
- (*meth*) shows how to do method lookup and returns a lambda with the method body, where the self variable is substituted by the instance.
- (*call*) defines how to call an object, by looking for the closest binding for the instance in order to get the closest binding for the class and its call method.
- (*else*) determines the behavior of an object when we try to access a member that is not present in the object.

4.2 Fw_{py}^{co} : core calculi for declarative OOP

Our source language is Fw_{py}^{co} , which extends Fw_{py} with copattern-based declarations. In Fw_{py}^{co} , we add a single new term: $\llbracket \overline{o} \approx_i \langle e ; \overline{s} \rangle \rrbracket$. This term represents a copattern matching capable object — a *co-object* — and carries a list (*order sensitive*) of observations to be checked against the incoming interactions, stored in the setting $\langle e ; \overline{s} \rangle$, while i tracks the current observation to be tested. These extra pieces of information are used to respect the declaration order of the observations while a

$$\begin{array}{c}
\text{def} \frac{bv(D) \notin FV(F)}{F[D; V] \mapsto D; F[V]} \\
\\
\text{new} \frac{y \notin FV(E) \quad C \notin BV(E)}{\text{class } C : (\text{init}(\text{self}, x) : \overline{a_i : e_i^i}; \overline{M}; \quad \text{class } C : (\text{init}(\text{self}, x) : \overline{a_i : e_i^i}; \overline{M};} \\
E[C(V)] \mapsto \text{new } y := C(\{\overline{a_i : e_i[V/x]^i}\}); \\
E[y] \\
\\
\text{attr} \frac{x \notin BV(E) \quad BV(E) \cap FV(V) = \emptyset}{\text{new } x := C(\{\overline{a_i : V_i^i}, a : V\}); \quad \text{new } x := C(\{\overline{a_i : V_i^i}, a : V\});} \\
E[x.a] \mapsto E[V] \\
\\
\text{else} \frac{C \notin BV(E_1, E_2) \quad x \notin BV(E_2) \quad a \notin \overline{M} \quad a \notin \{\overline{b : V}\}}{\text{class } C : I; ((\text{else}(\text{self}, y) : e); \overline{M}); \quad \text{class } C : I; ((\text{else}(\text{self}, y) : e); \overline{M});} \\
E_1 \left[\text{new } x := C(\{\overline{b : V}\}); \right] \mapsto E_1 \left[\text{new } x := C(\{\overline{b : V}\}); \right] \\
E_2[x.a] \mapsto E_2[e[x/\text{self}, 'a/y]] \\
\\
\text{meth} \frac{C \notin BV(E_1, E_2) \quad x \notin BV(E_2) \quad (m(\text{self}, y) : e) \in \overline{M} \quad m \notin \{\overline{b : V}\}}{\text{class } C : I; ((m(\text{self}, y) : e); \overline{M}); \quad \text{class } C : I; ((m(\text{self}, y) : e); \overline{M});} \\
E_1 \left[\text{new } x := C(\{\overline{b : V}\}); \right] \mapsto E_1 \left[\text{new } x := C(\{\overline{b : V}\}); \right] \\
E_2[x.m] \mapsto E_2[\lambda y. e[x/\text{self}]] \\
\\
\text{call} \frac{C \notin BV(E_1, E_2) \quad x \notin BV(E_2)}{\text{class } C : I; ((\text{call}(\text{self}, y) : e); \overline{M}); \quad \text{class } C : I; ((\text{call}(\text{self}, y) : e); \overline{M});} \\
E_1 \left[\text{new } x := C(\{\overline{b : V}\}); \right] \mapsto E_1 \left[\text{new } x := C(\{\overline{b : V}\}); \right] \\
E_2[x(\{\overline{l_i : V_i^i}\})] \mapsto E_2[e[x/\text{self}, \{\overline{l_i : V_i^i}\}/y]]
\end{array}$$

Fig. 11. Operational semantics for the class constructs of Fw_{py} .

copattern match is being performed. This new term is obtained using the new definition form, in which we define a class by a sequence of possible observations. An observation is an equation of the form $q = e$, where q is a copattern, and e is its outcome. A copattern $\langle \text{self } A ; \bar{r} \rangle$ pairs a root parameter pattern A , which constrains the constructor arguments, with a (possibly empty) sequence of steps $\bar{r}$. A step can be an attribute access ($\square.a$) or a call specifying the shape of its argument ($\square A$). These requirements are created by patterns, which can be an always satisfying variable pattern, a literal pattern, or a parameter pattern. A parameter pattern can be a strict pattern $((\overline{l_i : p_i^i}))$, requiring the exact shape and size, or a row-dictionary pattern $(\{\overline{l_i : p_i^i}\})$, having a flexible (at least) shape. A setting $\langle V ; \bar{s} \rangle$ has the same list structure, but holds a value V as its head and actual interaction steps $\bar{s}$ rather than patterns.

The dynamics of the new constructs and the copattern match are defined in terms of the evaluation contexts, frames, and settings. These settings allow us to store interactions performed on an object and are used to check whether we are compatible with an observation. The mechanics of the copattern matching are defined by three new reduction rules, which are triggered when we perform an interaction on a $\{\bar{o} \approx_i \langle e ; \bar{s} \rangle\}$ term. These rules use an auxiliary function, $\approx$ fig. 14, to check if

$$\begin{aligned}
& \dots \\
& \text{Expression} \ni e ::= \dots \mid \llbracket \bar{O} \approx_i \langle e ; \bar{s} \rangle \rrbracket \\
& \text{Value} \ni V ::= \dots \mid \llbracket \bar{O} \approx_i \langle V ; \bar{s} \rangle \rrbracket \\
& \text{Definition} \ni D ::= \dots \mid \text{dec } C : \bar{O} \\
& \text{Observation} \ni O ::= q = e \\
& \text{QStep} \ni r ::= \square.a \mid \square A \\
& \text{Copattern} \ni q ::= \langle \text{self } A ; \bar{r} \rangle \\
& \text{ParamPat} \ni A ::= (\bar{a}_i : \bar{p}_i^i) \mid \{\bar{a}_i : \bar{p}_i^i, \dots x\} \\
& \text{Pattern} \ni p ::= x \mid l \mid A \\
& \text{SStep} \ni s ::= \square.a \mid \square(V) \\
& \text{Setting} \ni t ::= \langle \square ; \bar{s} \rangle
\end{aligned}$$

Fig. 12. Syntax of Fw_{py}^{co} — Declarative classes.

the current interaction is compatible with the i -th observation, and to extract the variable bindings if it is the case.

- (dec) tells us how to create a new co-object by fetching the observations from the class declaration.
- (full-match) specifies the behavior of a co-object when it receives an interaction that fully satisfies the current observation. If the extended setting of the interaction is compatible with the i -th observation, then we return the outcome of the observation, plugging the bindings produced by $\approx$ and **self**.
- (part-match) determines the behavior of a co-object when it receives an interaction that partially satisfies the current observation; in this case, the co-object should update its state to reflect the new information.
- (fail-match) defines the behavior of a co-object when it receives an interaction that does not satisfy the current observation; in this case, the co-object should move on to the next observation, by incrementing i , and retry the interactions to see if our matching context is satisfied by the next observation.³

We assume that there are no repeated free variables in a (co)pattern. To illustrate, $\langle \text{self } (x : p) ; \square.m, \square (x : p') \rangle$ is not a valid copattern, since x is repeated. We can rewrite it as $\langle \text{self } (x : p) ; \square.m, \square (y : p') \rangle$, which is valid.

A setting $\langle \square ; \bar{s} \rangle$ can be used as an evaluation context by filling its hole with an expression e :

$$\langle \square ; \rangle[e] \stackrel{\text{def}}{=} e \quad \langle \square ; \bar{s}, s' \rangle[e] \stackrel{\text{def}}{=} s'[\langle \square ; \bar{s} \rangle[e]]$$

That is, the steps are applied left-to-right, producing the evaluation-context chain $s_n[\dots[s_1[e]]\dots]$.

The operations defined in fig. 14 return one of the three results: **fail** on a failure, **partial** σ on a partial match, and **match** σ on a successful match, where σ is a collection of substitutions. The operation $\approx$ performs the copattern match. It checks the lengths of the copattern and the setting,

³We don't model stuck states, here if we run out of observations, we obtain a stuck value.

$$\begin{array}{c}
\vdots \\
\text{dec} \frac{D = \mathbf{dec} C : \overline{O} \quad C \notin BV(E)}{D; E[C(V)] \mapsto D; E[\{\{\overline{O} \approx_0 \langle V ; \rangle\}\}]} \\
\text{full-match} \frac{\overline{O} = \overline{O}_i^i, (q = e), \overline{O}_j^j \quad q \approx \langle V ; \bar{s}, s \rangle = \mathbf{match} \sigma}{s[\{\{\overline{O} \approx_i \langle V ; \bar{s} \rangle\}\}] \mapsto e[\{\{\overline{O} \approx_0 \langle V ; \rangle\}\} / \mathbf{self}, \sigma]} \\
\text{part-match} \frac{\overline{O} = \overline{O}_i^i, (q = e), \overline{O}_j^j \quad q \approx \langle V ; \bar{s}, s \rangle = \mathbf{partial} \sigma}{s[\{\{\overline{O} \approx_i \langle V ; \bar{s} \rangle\}\}] \mapsto \{\{\overline{O} \approx_i \langle V ; \bar{s}, s \rangle\}\}} \\
\text{fail-match} \frac{\overline{O} = \overline{O}_i^i, (q = e), \overline{O}_j^j \quad q \approx \langle V ; \bar{s}, s \rangle = \mathbf{fail}}{s[\{\{\overline{O} \approx_i \langle V ; \bar{s} \rangle\}\}] \mapsto s[\langle \square ; \bar{s} \rangle[\{\{\overline{O} \approx_{i+1} \langle V ; \rangle\}\}]]}
\end{array}$$

Fig. 13. Operational semantics for Fw_{py}^{Co} .

and whether they are point-wise compatible. For each individual pair of copattern and setting element, we use $\smile$ to check whether they are compatible, and $\triangleright$ to perform pattern matching.

The relation $\approx$ will return the substitutions together with **match** if the copattern and the setting are fully compatible, **partial** if the copattern is longer than the setting, and **fail** otherwise. For $\smile$, we check each individual step-pair compatibility, and we return the substitutions if they are compatible, and **fail** otherwise. Attributes copattern checks if the setting step is an attribute access with the same name, and call copattern check if the setting step is a call with an argument compatible with the parameter pattern. If they are not compatible or have a mismatch in the copattern and setting step types, we return **fail**. Lastly, $\triangleright$ checks if a pattern is compatible with a value, and returns the substitutions if they are compatible, and **fail** otherwise. Variable patterns are compatible with any value, literal patterns are compatible with values that are equal to them, and parameter patterns are compatible with dictionaries that have the required keys and compatible values. For strict parameter patterns, we also ensure the dictionary does not contain extra keys, whereas for row-dictionary patterns, we allow extra keys.

The intuition behind the (co)pattern match relation is that we can transform copatterns into settings by substituting all its free variables for concrete values. We capture this with lemma 4.1, whose proof follows by induction on the structure of the (co)pattern.

LEMMA 4.1 (SPECIFICATION OF (CO)PATTERN MATCHING).

- (a) $p \triangleright V' = \mathbf{match} \overline{V/x} \text{ iff } p[\overline{V/x}] = V' \text{ and } \{\bar{x}\} = FV(p).$
- (b) $q \approx t[V] = \mathbf{match} \overline{V_l/x_l^l} \text{ iff } q[\overline{V_l/x_l^l}] = t[V], \text{ and } \{\bar{x}_l^l\} = FV(q).$
- (c) $\langle \mathbf{self} A ; \bar{r}^n, r \rangle \approx \langle V ; \bar{s}^n \rangle = \mathbf{partial} \overline{V_l/x_l^l} \text{ iff } \langle \mathbf{self} A ; \bar{r}^n \rangle \approx \langle V ; \bar{s}^n \rangle = \mathbf{match} \overline{V_l/x_l^l}.$

5 From CoPy to Featherweight Python: Translating Away Copatterns

We model our implementation of copatterns in Python as a translation from Fw_{py}^{Co} to Fw_{py} . For this translation we use a prelude environment⁴ containing the classes representing our new constructs and operations. The key point is the role of **call** and **else** methods. The base *CoData* class provides implementation for both of them, and each declaration translation just specialize the constructor

⁴The full prelude definition is in the section A

Copattern matching:

$$\begin{array}{c}
\frac{\forall k \leq n. r_k \smile s_k = \mathbf{match} \sigma_k \quad A \triangleright \{\overline{l} :: \overline{V}\} = \mathbf{match} \sigma}{\langle \mathbf{self} A ; \overline{r}^n \rangle \approx \langle \{\overline{l} :: \overline{V}\} ; \overline{s}^n \rangle = \mathbf{match} \sigma, \overline{\sigma}_k^k} \\
\\
\frac{n > m \quad \forall k \leq m. r_k \smile s_k = \mathbf{match} \sigma_k \quad A \triangleright \{\overline{l} :: \overline{V}\} = \mathbf{match} \sigma}{\langle \mathbf{self} A ; \overline{r}^n \rangle \approx \langle \{\overline{l} :: \overline{V}\} ; \overline{s}^m \rangle = \mathbf{partial} \sigma, \overline{\sigma}_k^k} \\
\\
\frac{n < m}{\langle \mathbf{self} A ; \overline{r}^n \rangle \approx \langle \{\overline{l} :: \overline{V}\} ; \overline{s}^m \rangle = \mathbf{fail}} \\
\\
\frac{A \triangleright \{\overline{l} :: \overline{V}\} = \mathbf{match} \sigma \quad \exists k \leq \min(n, m). r_k \smile s_k = \mathbf{fail}}{\langle \mathbf{self} A ; \overline{r}^n \rangle \approx \langle \{\overline{l} :: \overline{V}\} ; \overline{s}^m \rangle = \mathbf{fail}} \\
\\
\frac{A \triangleright V = \mathbf{fail}}{\langle \mathbf{self} A ; \overline{r} \rangle \approx \langle V ; \overline{s} \rangle = \mathbf{fail}} \\
\\
\frac{}{\square.a \smile \square.a = \mathbf{match} /} \quad \frac{a \neq b}{\square.a \smile \square.b = \mathbf{fail}} \quad \frac{s \neq \square.b}{\square.a \smile s = \mathbf{fail}} \\
\\
\frac{A \triangleright V = \mathbf{match} \sigma}{\square A \smile \square(V) = \mathbf{match} \sigma} \quad \frac{A \triangleright V = \mathbf{fail}}{\square A \smile \square(V) = \mathbf{fail}} \quad \frac{s \neq \square(V)}{\square A \smile s = \mathbf{fail}}
\end{array}$$

Pattern matching:

$$\begin{array}{c}
\frac{}{x \triangleright V = \mathbf{match} V/x} \quad \frac{l = l'}{l \triangleright l' = \mathbf{match} /} \quad \frac{V \neq l}{l \triangleright V = \mathbf{fail}} \\
\\
\frac{\forall i. p_i \triangleright V_i = \mathbf{match} \sigma_i}{(\overline{a_i} : p_i^i) \triangleright \{\overline{a_i} : \overline{V_i}^i\} = \mathbf{match} \overline{\sigma_i}^i} \quad \frac{i \neq j}{(\overline{a_i} : p_i^i) \triangleright \{\overline{a_j} : \overline{V_j}^j\} = \mathbf{fail}} \\
\\
\frac{\exists i. p_i \triangleright V_i = \mathbf{fail}}{(\overline{a_i} : p_i^i) \triangleright \{\overline{a_i} : \overline{V_i}^i\} = \mathbf{fail}} \quad \frac{V \neq \{\overline{b_j} : \overline{V_j}^j\}}{(\overline{a_i} : p_i^i) \triangleright V = \mathbf{fail}} \\
\\
\frac{\forall i. p_i \triangleright V_i = \mathbf{match} \sigma_i}{\{\overline{a_i} : p_i^i, \dots x\} \triangleright \{\overline{a_i} : \overline{V_i}^i, \overline{b_j} : \overline{V_j}^j\} = \mathbf{match} \overline{\sigma_i}^i, \{\overline{b_j} : \overline{V_j}^j\}/x} \\
\\
\frac{\exists i. p_i \triangleright V_i = \mathbf{fail}}{\{\overline{a_i} : p_i^i, \dots x\} \triangleright \{\overline{a_i} : \overline{V_i}^i, \overline{b_j} : \overline{V_j}^j\} = \mathbf{fail}} \quad \frac{V \neq \{\overline{b_j} : \overline{V_j}^j\}}{\{\overline{a_i} : p_i^i, \dots x\} \triangleright V = \mathbf{fail}}
\end{array}$$

Fig. 14. (Co)Pattern match specification.

for its observation list. Thus, the image of co-objects doesn't contain attributes, so every attribute access happens to be an *else* case. We can see the translation of the new constructs in fig. 15. The missing cases are just recursive calls to the translation function, since they don't involve any of the new constructs.

$$\boxed{\llbracket _ \rrbracket : Fw_{p_y}^{c_o} \rightarrow Fw_{p_y}}$$

Translation of declarations, co-objects, and observations:

$$\begin{aligned} \llbracket \text{dec } C : \overline{O_i}^i \rrbracket &= \text{class } C(\text{CoData}) : \text{init}(x) : \text{obvs} = \llbracket \overline{O_i}^i \rrbracket ; \text{st} = [T\text{Self}(x)] ; i = 0 \\ \llbracket \overline{O_i}^i \approx_i \langle V ; \bar{s} \rangle \rrbracket &= \text{CoData}(\llbracket \overline{O_i}^i \rrbracket, \llbracket \langle V ; \bar{s} \rangle \rrbracket, i) \\ \llbracket q = e \rrbracket &= \text{Obs}(\llbracket q \rrbracket, \lambda(\text{self}, y). \overline{\text{let } x_i := y[x_i] \text{ in } [e]})^i \end{aligned} \quad (\text{where } BV(q) = \{\bar{x}\})$$

Translation of copatterns:

$$\llbracket \langle \text{self } A ; \bar{r} \rangle \rrbracket = [Q\text{Self}(\llbracket A \rrbracket)] ++ \llbracket \bar{r} \rrbracket$$

Translation of QSteps:

$$\begin{aligned} \llbracket \square.a \rrbracket &= Q\text{Attr}('a) \\ \llbracket \square A \rrbracket &= Q\text{App}(\llbracket A \rrbracket) \end{aligned}$$

Translation of settings:

$$\llbracket \langle V ; \bar{s} \rangle \rrbracket = [T\text{Self}(\llbracket V \rrbracket)] ++ \llbracket \bar{s} \rrbracket$$

Translation of SSteps:

$$\begin{aligned} \llbracket \square.a \rrbracket &= T\text{Attr}('a) \\ \llbracket \square(V) \rrbracket &= T\text{App}(\llbracket V \rrbracket) \end{aligned}$$

Translation of patterns:

$$\begin{aligned} \llbracket x \rrbracket &= P\text{Var}('x) \\ \llbracket l \rrbracket &= P\text{Lit}(l) \\ \llbracket \overline{(l_i : p_i)}^i \rrbracket &= P\text{Dict}(\{\overline{l_i : [p_i]}^i\}) \\ \llbracket \overline{\{l_i : p_i, \dots a\}}^i \rrbracket &= P\text{Row}(\{\overline{l_i : [p_i]}^i\}, 'a) \end{aligned}$$

Fig. 15. Translation of the new features of $Fw_{p_y}^{c_o}$ to Fw_{p_y} .

5.1 Prelude

The bulk of our translation is on the implementation of (co)-pattern match. Each declaration is translated to a plain class specializing the *CoData*'s constructor. The class *CoData* stores a list of observations and the current setting (matching context). As mentioned, it starts a comatch, represented by the method *comatch*, with the implementations of *call* and *else*. The *CoData.comatch* captures the semantics of the $Fw_{p_y}^{c_o}$ reduction rules.

```
comatch(self, args) :
  if self.i ≥ len(self.obvs) then STUCK
  else let r = self.obvs[self.i].comatch(self) in
    if r ≠ None then r
    else let bb' = CoData(self.obvs, [self.st[0]], self.i+1) in
      fold(λa, f.f.unwind(a), drop(1, self.st), bb')
```

In this method, we fetch the i -th observation o and call $o.comatch(\text{self})$, which now returns a binding dictionary r or **None**. If $r = \text{None}$ the observation fails: we create a new co-object bb' with a bumped index and a reset setting, then replay the accumulated frames via *fold* (the unwind step), otherwise we get **STUCK**. If $r \neq \text{None}$ and the copattern is longer than the current setting

($\text{len}(o.q) > \text{len}(\text{self}.st)$), we have a partial match and return **self** unchanged, waiting for more interactions. Otherwise, the lengths are equal, and we have a full match: we commit by calling $o.out(\text{self}, r)$ with the produced bindings.

Each observation holds a copattern and a lambda representing its outcome. An observation only has a single method: *comatch*. This method implements the operation $\approx$ defined in fig. 14. This is a total function returning **None** on failure, the co-object *bb* on a partial match, or the outcome value on a full match. Each individual copattern fragment knows how to respond to *comatch* and implements $\sim$, using **None** for failure and a dictionary of bindings for success. Patterns follow a similar structure, responding to *match* and implementing $\triangleright$.

```
comatch(self, bb) : let f = λsub, k. let r = self.q[k].comatch(bb.st[k]) in
  if sub= None ∨ r= None then None else sub ++ r in
  if len(self.q) > len(bb.st) then fold(f, bb.st, {})
  else if len(self.q)=len(bb.st) then let sub = fold(f, self.q, {}) in
    if sub= None then None else self.out(bb, sub)
  else None
```

To guarantee that this encoding of copatterns is faithful to our semantics we proof a forward simulation between $Fw_{p_y}^{co}$ terms and their respective translations. This means, that we have shown that if a term M can take a step to N in $Fw_{p_y}^{co}$, the translation of M will eventually reach the translation of N .

THEOREM 5.1 ($\llbracket _ \rrbracket$ FORWARDS SIMULATION). *If $M \rightarrow N$ in $Fw_{p_y}^{co}$, then there exists a Fw_{p_y} term T such that $\llbracket M \rrbracket \rightarrow^* T$ and $T = \llbracket N \rrbracket$ (modulo observational equivalence).*

PROOF. This follows from lemmas showing that each $Fw_{p_y}^{co}$ reduction is simulated by $\llbracket _ \rrbracket$ modulo observational equivalence. $\square$

These auxiliary lemmas depend upon the following properties stating that our translation in fact preserves the meaning of our operations:

LEMMA 5.2 ($\llbracket _ \rrbracket$ PRESERVES PATTERN MATCHING). (a) $p \triangleright V = \text{match } \overline{V/x} \text{ iff } \llbracket p \rrbracket.match(\llbracket V \rrbracket) \rightarrow^* \{x : \llbracket V \rrbracket\}$.
 (b) $p \triangleright V = \text{fail} \text{ iff } \llbracket p \rrbracket.match(\llbracket V \rrbracket) \rightarrow^* \text{None}$.

PROOF. By induction on the structure of p . $\square$

LEMMA 5.3 ($\llbracket _ \rrbracket$ PRESERVES COPATTERN MATCHING). *Let bb be a co-object with $bb.st = \llbracket \langle V ; \bar{s} \rangle \rrbracket$.*

(a) $q \approx \langle V ; \bar{s} \rangle = \text{match } \overline{V/x} \text{ or } q \approx \langle V ; \bar{s} \rangle = \text{partial } \overline{V/x} \text{ iff } \llbracket q \rrbracket.comatch(bb) \rightarrow^* \{x : \llbracket V \rrbracket\}$.
 (b) $q \approx \langle V ; \bar{s} \rangle = \text{fail} \text{ iff } \llbracket q \rrbracket.comatch(bb) \rightarrow^* \text{None}$.

PROOF. By induction on the structure of q . $\square$

6 Implementation Details & Challenges

At first glance, the semantics of those operations can look alien to seasoned Pythonistas. But in reality, this behavior is encoded as regular Python classes with heavy use of method overloading. As we can see in fig. 16, each declaration is transformed into a Python class with the required metadata to perform copattern matching. However, the real implementation is more involved than $\llbracket _ \rrbracket$.

```

class Zeroes_handlers:
    def _head_____(self,):
        return 0
    def _tail_____(self,):
        return self

class Zeroes(CoData):
    __allowed_names__ = {'head', 'tail'}
    __metadata__ = Dec('Zeroes', ['CoData'],
        [Observation([ESelf(), EAttr(name='head'),
            ↪ ECall(fields=[]),
            Zeroes_handlers._head_____),
        Observation([ESelf(), EAttr(name='tail'),
            ↪ ECall(fields=[]),
            Zeroes_handlers._tail_____)])])

dec Zeroes:
    self().head() = 0
    self().tail() = self

```

(a) Originating *dec* statement. (b) Generated class.

Fig. 16. Example of translation of a *dec* statement.

Instead of generating lambdas for each observation, we generate methods in an auxiliary class, and store references to them in the main class. We have this extra class because Python’s lambda don’t support statements, and we want to allow users to write statements in their observation’s outcomes. This approach enables us to leverage Python’s self parameter, which can be explicitly overridden when using a method reference. The current implementation translates a copattern to a method named, we could have opted for a more opaque name, like indices or hashes, but we found that this approach is more debuggable and easier to understand.

In Fw_{py} , we use a very simple approach to inheritance, where we only allow single inheritance, and we don’t have a way to call the super methods. This streamlines inheritance to appending the methods of the parent class into the child class. CoPy embraces Python’s multiple inheritance, so we need to be careful with the method resolution order and inherited members. To handle this, we store extra bits of metadata in the main generated class, such as the allowed names to start a copattern match (`__allowed_names__`) and the information of each observation.

6.1 Copattern matching: Underneath the abstraction

All the copattern matching functionality is defined in the CoData class. In this class, we override the `__getattr__` special method — this can be interpreted as a “.” overload—. So after each attribute access, we check if it is a name that can start a copattern match. If it is, we return a copattern match object that holds a representation of the current context and a reference to its co-object. The comatch object handles the rest of the copattern matching. Changes to our evaluation context can be captured by overriding the `__getattr__` and `__call__` methods. With those, we can capture all members’ access, calls, and their arguments in a context. So, during a comatch, after each change, we test if our current context is compatible with one of the copatterns inside our co-object. This check respects the declaration order and tries to match the sequence of method calls in our context with a copattern. Thus, every method name must be exactly equal to the name in the copattern, and for the arguments, we need to check if the argument satisfies the pattern in the copattern. We use a generic constructor for every declaration which stores its positional and named constructor arguments to be used in the copattern matching.

One important remark is what happens in a partial match. When we have an incomplete context, we return a CoMatch object. This intermediate value could be used in the future in a larger context.

So, if we try to use this intermediate result as a value, we need to force this CoMatch into an outcome. Happily, we mimic this behavior by overloading way too many special methods, such as `__eq__`, `__str__`, and `__repr__`. So, for each way we can interact with an object, we will do a comatch that does not accept an intermediate result. Those overloads are in the CoDataDunder class, which CoData inherits.

Our theoretical model (fig. 17) re-triggers all accumulated interactions from scratch on a fail by folding over the tail of the setting. The real implementation avoids this redundant replay through a prefix-match optimization: if the copattern is shorter than the current setting and all its fragments match, the outcome is committed immediately, and only the remaining interactions are reapplied onto the result. This short-circuits the fail-match cascade without changing the observable semantics.

6.2 Wait, I thought Python did not have macros!

CoPy defines new syntax for Python, yet it is distributed as a plain package rather than a language fork or an external code-generation tool. This is possible because Python exposes metaprogramming hooks deep in its import and codec machinery. We have implemented a compiler that transforms a `dec` statement into its respective Python class, and two entry points to trigger this translation; their usage is described in section 3.

The first entry point is an *import hook* (`loader.py`). Python’s import system allows custom behavior when loading a module by registering an object on `sys.meta_path`. We use such a hook to run our parser on `.copy` files: the parser returns each declaration’s metadata together with the generated plain-Python source, which is then handed directly to Python’s bytecode compiler.

The second entry point is a *custom text codec* (`copycodec.py`). Python allows user-defined codecs that intercept the bytes-to-string step performed when reading any source file. We exploit this to run our parser transparently on regular `.py` files that declare `# -*- coding: copy -*-` at the top, requiring no change to file extensions or import statements. The codec is registered at interpreter startup via a `.pth` file placed in *site-packages*: Python processes `.pth` files before any user code runs, so the codec is always available once installed.

7 Related Work

Copatterns and Codata. Copattern matching [2] is the dual of pattern matching [19]. While pattern matching allows us to break down values and reason about each possible case, copattern matching enables us to synthesize objects through a finite set of observations, *i.e.*, copattern matching equalities. This follows the same duality between data and codata. While data types are portrayed by the definition of constructors to model finite tree-shaped structures, codata types are described by the use of destructors with potentially infinite entities. In other words, codata types are categorized by how we can interact with their values instead of spelling out exactly how they are made. For example, the common example of an infinite stream is a codata type that needs to respond to two destructors: *head* and *tail*. The observation *head* should return the first element in the stream, and the observation *tail* should return a stream without the first element.

Copattern Implementations. Even with this observational nature of codata, copatterns are most often found within the functional realm. Copattern matching has its most mature implementation in Agda [3]. However, since Agda is a proof assistant, it imposes a restrictive well-foundedness check on copattern definitions to ensure termination — a constraint not relevant for ordinary programming tasks. Similarly, Beluga [14, 15] supports copatterns in a dependently-typed proof-assistant setting. In Racket, CoScheme [6] focuses on composable copatterns, exploiting the dynamic environment and Racket’s macro system to support new forms of declaration composition.

Laforge and Régis-Gianas [13] implement copatterns as a macro extension for OCaml. Their translation maps codata declarations to OCaml record types and copattern clauses to cases in a function defined by ordinary pattern matching, yielding a purely *functional* encoding. Our translation `[_]` takes a different route: each declaration becomes a class inheriting from *CoData*, which dispatches copattern matching through the *call* and *else* special methods. This *class-based* encoding is intentional — it mirrors the object-oriented nature of the target language and highlights the *call/else* duality that is central to the codata–OOP correspondence. OCaml’s copattern extension remains unofficial and has not yet been merged into the main implementation.

Objects and Codata. Cook [4] drew a sharp distinction between abstract data types (ADTs) and objects: while ADTs are characterized by how they are *constructed*, objects are characterized by how they are *observed*. This is precisely the data/codata duality. Objects, in Cook’s sense, are codata types whose interface consists of the observations one may perform on them. Downen et al. [7] make this correspondence explicit and demonstrate that codata can be compiled into OOP constructs across eager, lazy, functional, and object-oriented host languages. CoPy builds directly on this line of work: rather than using the codata–OOP correspondence only as a compilation technique, we make it *user-facing* by exposing copattern syntax directly in Python.

The earlier theoretical foundation of Abadi and Cardelli [1] defines a calculus of objects in terms of the λ -calculus, providing a basis for reasoning about OOP programs by translating them into a functional setting. CoPy pursues the orthogonal direction: rather than compiling OOP into functional calculi *after* the fact for the purpose of external reasoning, we bring functional-style equational reasoning *inside* OOP through copattern syntax, enabling programmers to reason about their code in place.

Language Extension and Distribution. Our strategy of distributing CoPy as a plain Python package rather than a language fork follows the spirit of Languages as Libraries [16] and Racket’s macro system [8]. In those settings, new language constructs are defined as library code using a powerful hygienic macro system, making extensions indistinguishable from built-in notation. Python provides no Lisp-style macro facilities, so CoPy instead intercepts Python’s module loading and text decoding pipelines via an import hook and a custom codec. This allows CoPy to be installed with pip and used without patching the interpreter, achieving a similar level of accessibility.

8 Future Work & Conclusion

Richer Copattern Language. The current copattern language of CoPy supports observations on attribute access, method calls, and argument shapes. A natural extension is to add *combinators* that allow copattern fragments to be composed and repeated, enabling more concise declarations for objects with regular observation structure. We also plan to support guards in copatterns and better interaction with Python’s built-in operators within copattern clauses.

Traits, Coverage, and Interfaces. Due to Python’s dynamic typing, the current version of CoPy has no notion of interfaces and performs no coverage or typing checks on copattern declarations. This aligns with Python’s own approach to pattern matching, which likewise imposes no static checks without an external tool such as MyPy or Pyright. Nevertheless, we plan to introduce *CoPy interface declarations* that specify the observations an object must handle. These interfaces would serve two purposes: checking that a set of copatterns covers all required observations, and verifying that composed declarations satisfy a given interface.

Type System & Other Target Languages. The theoretical model presented in this paper is intentionally untyped, leaving open the design of a type system for CoPy. A typed extension would make the codata–OOP correspondence more precise, allowing copattern declarations to be checked

against interface types at compile time. We view the design of such a type system as a natural follow-up to this work. We would like to validate the copattern encoding in languages beyond Python, particularly in statically typed settings with strong metaprogramming support. Lean4 [5] and Rhombus [8] are strong candidates: both provide class-like constructs and expressive macro systems that should accommodate the class-based translation at the core of CoPy.

In this work, we have shown how to use and implement copatterns to enable a declarative flavor of object-oriented programming. Copatterns permit programmers to write code closely resembling the way they would equationally describe the behavior of objects regarding their surroundings. This nested nature of copatterns empowers a rich, expressive, and direct way to specify the behavior of objects, while using these specifications as a tool for reasoning about their behavior. To capture the essence of this style of programming, and to explain how it can be implemented, we have designed a core language, Fw_{py}^{co} , and a translation to a class-based language Fw_{py} . This translation captures the semantics of copatterns and calls attention to the key features of the target language that are required to support copatterns, such as classes, objects, and overloading. The core language Fw_{py}^{co} , the translation to Fw_{py} , and the CoPy implementation together demonstrate that the codata–OOP correspondence is not merely a theoretical observation but a practical foundation for bringing equational, declarative programming into mainstream object-oriented languages.

Data Availability Statement

CoPy is a Python language extension distributed as a dynamically-loaded module and a pre-processor. The implementation is available at⁵, and it is included in the supplementary materials and will be submitted for the Artifact Evaluation process. The repository contains the source code, installation and usage instructions, and illustrative examples demonstrating its functionality.

References

- [1] Martin Abadi and Luca Cardelli. 1996. *A Theory of Objects. Monographs in Computer Science*. Springer, New York. ISBN: 978-0-387-94775-4.
- [2] Andreas Abel, Brigitte Pientka, David Thibodeau, and Anton Setzer. 2013. Copatterns: programming infinite structures by observations. In *Proceedings of the 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '13)*. ACM, Rome, Italy, 27–38. ISBN: 978-1-4503-1832-7. doi:10.1145/2429069.2429075.
- [3] Jesper Cockx and Andreas Abel. 2018. Elaborating dependent (co)pattern matching. *Proceedings of the ACM on Programming Languages*, 2, ICFP, Article 75, 30 pages. doi:10.1145/3236770.
- [4] William R. Cook. 1990. Object-oriented programming versus abstract data types. In *Proceedings of the REX School/Workshop on Foundations of Object-Oriented Languages*. Springer-Verlag, Berlin, Heidelberg, 151–178. ISBN: 354053931X.
- [5] Leonardo de Moura and Sebastian Ullrich. 2021. The lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12–15, 2021, Proceedings*. Springer-Verlag, Berlin, Heidelberg, 625–635. ISBN: 978-3-030-79875-8. doi:10.1007/978-3-030-79876-5_37.
- [6] Paul Downen and Adriano Corbelino II. 2025. Coscheme: compositional copatterns in scheme. In *Trends in Functional Programming: 26th International Symposium, TFP 2025, Oxford, UK, January 14–16, 2025, Revised Selected Papers*. Springer-Verlag, Oxford, United Kingdom, 205–258. ISBN: 978-3-031-99750-1. doi:10.1007/978-3-031-99751-8_10.
- [7] Paul Downen, Zachary Sullivan, Zena M. Ariola, and Simon Peyton Jones. 2019. Codata in action. In *Programming Languages and Systems - 28th European Symposium on Programming, ESOP 2019, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2019 (ESOP '19)*. Springer International Publishing, Prague, Czech Republic, 119–146. ISBN: 978-3-030-17184-1. doi:10.1007/978-3-030-17184-1_5.
- [8] Matthew Flatt et al. 2023. Rhombus: a new spin on macros without all the parentheses. *Proc. ACM Program. Lang.*, 7, OOPSLA2, Article 242, (Oct. 2023), 30 pages. doi:10.1145/3622818.
- [9] Martin Fowler. 2005. Fluent interface. <https://martinfowler.com/bliki/FluentInterface.html>. Accessed: 2026-03-14. (Dec. 2005).
- [10] Zhenjiang Hu, John Hughes, and Meng Wang. 2015. How functional programming mattered. *National Science Review*, 2, 3, (Sept. 1, 2015), 349–370. doi:10.1093/nsr/nwv042.

⁵https://osf.io/yqub3/overview?view_only=0a16f85d924a431eb40d1da4c1406cd6

- [11] John Hughes. 1989. Why functional programming matters. *The Computer Journal*, 32, 2, 98–107. doi:[10.1093/comjnl/32.2.98](https://doi.org/10.1093/comjnl/32.2.98).
- [12] Atsushi Igarashi, Benjamin C. Pierce, and Philip Wadler. 2001. Featherweight java: a minimal core calculus for java and gj. *ACM Transactions on Programming Languages and Systems*, 23, 3, 396–450. doi:[10.1145/503502.503505](https://doi.org/10.1145/503502.503505).
- [13] Paul Laforgue and Yann Régis-Gianas. 2017. Copattern matching and first-class observations in OCaml, with a macro. In *Proceedings of the 19th International Symposium on Principles and Practice of Declarative Programming, Namur, Belgium, October 09 - 11, 2017*, 97–108. doi:[10.1145/3131851.3131869](https://doi.org/10.1145/3131851.3131869).
- [14] Brigitte Pientka. 2010. Beluga: programming with dependent types, contextual data, and contexts. In *Proceedings of the 10th International Conference on Functional and Logic Programming (FLOPS'10)*. Springer-Verlag, Sendai, Japan, 1–12. ISBN: 3642122507. doi:[10.1007/978-3-642-12251-4_1](https://doi.org/10.1007/978-3-642-12251-4_1).
- [15] David Thibodeau. 2020. *An Intensional Type Theory of Coinduction using Copatterns*. Ph.D. Dissertation. McGill University, Montreal, QC, Canada, (Dec. 2020).
- [16] Sam Tobin-Hochstadt, Vincent St-Amour, Ryan Culpepper, Matthew Flatt, and Matthias Felleisen. 2011. Languages as libraries. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '11)*. Association for Computing Machinery, San Jose, California, USA, 132–141. ISBN: 9781450306638. doi:[10.1145/1993498.1993514](https://doi.org/10.1145/1993498.1993514).
- [17] P Wadler. 1987. A critique of Abelson and Sussman or why calculating is better than scheming. *SIGPLAN Not.*, 22, 3, (Mar. 1, 1987), 83–94. doi:[10.1145/24697.24706](https://doi.org/10.1145/24697.24706).
- [18] Mitchell Wand. 1991. Type inference for record concatenation and multiple inheritance. *Information and Computation*. Selections from 1989 IEEE Symposium on Logic in Computer Science 93, 1, (July 1, 1991), 1–15. doi:[10.1016/0890-5401\(91\)90050-C](https://doi.org/10.1016/0890-5401(91)90050-C).
- [19] Noam Zeilberger. 2008. On the unity of duality. *Annals of Pure and Applied Logic*, 153, 1, 660–96. doi:[10.1016/j.apal.2008.01.001](https://doi.org/10.1016/j.apal.2008.01.001).

```

class CoData :
  init(obvs, st, i) : self.obvs = obvs; self.st = st; self.i = i;
  call(self, args) :
    CoData(self.obvs, self.st ++ [TApp(args)], self.i).comatch({});
  comatch(self, args) :
    if self.i ≥ len(self.obvs) then STUCK
    else let o = self.obvs[self.i], r = o.comatch(self) in
      if r = None then let bb' = CoData(self.obvs, [self.st[0]], self.i+1) in fold(λa, f.f.unwind(a), drop(1, self.st), bb')
      else if len(o.q) > len(self.st) then self
      else o.out(self, r);
  else(self, a) : if self.st[0].v ? a then self.st[0].v[a]
    else CoData(self.obvs, self.st ++ [TAttr(a)], self.i).comatch({})

class Obs :
  init(q, out) : self.q = q; self.out = out;
  comatch(self, bb) : let f = λsub, k. let r = self.q[k].comatch(bb.st[k]) in if sub = None ∨ r = None then None else sub ++ r in
    if len(self.q) > len(bb.st) then fold(f, bb.st, {})
    else if len(self.q) = len(bb.st) then fold(f, self.q, {})
    else None

class QSelf : init(p) : self.p = p; comatch(self, t) : if isinstance(t, TSelf) then self.p.match(t.v) else None
class QAttr : init(a) : self.a = a; comatch(self, t) : if isinstance(t, TAttr) ∧ t.a = self.a then {} else None
class QApp : init(params) : self.params = params;
  comatch(self, t) : if isinstance(t, TApp) then self.params.match(t.args) else None

class PVar : init(name) : self.name = name; match(self, v) : {self.name : v}
class PLit : init(l) : self.l = l; match(self, v) : if self.l = v then {} else None
class PDict : init(pats) : self.pats = pats;
  match(self, v) : if ¬isinstance(v, dict) ∨ keys(v) ≠ keys(self.pats) then None
  else let f = λsub, k. let r = self.pats[k].match(v[k]) in if sub = None ∨ r = None then None else sub ++ r in
    fold(f, self.pats, {})

class PRow : init(pats, r) : self.pats = pats; self.r = r;
  match(self, v) : let f = λsub, k. let r = self.pats[k].match(v[k]) in
    if sub = None ∨ r = None then None else sub ++ r in
    fold(f, self.pats, {self.r : v - self.pats})

class TSelf : init(v) : self.v = v
class TAttr : init(a) : self.a = a; unwind(self, e) : e.self.a
class TApp : init(args) : self.args = args; unwind(self, e) : e(self.args)

```

Fig. 17. The copattern-matching infrastructure of Fw_{py}^{co} as Fw_{py} classes.

A Fw_{py} Prelude

B Proofs

B.1 (Co)Pattern Matching

B.1.1 Properties of (Co)Pattern Matching.

LEMMA 4.1 (SPECIFICATION OF (CO)PATTERN MATCHING).

- (a) $p \triangleright V' = \mathbf{match} \overline{V/x}$ iff $p[\overline{V/x}] = V'$ and $\{\bar{x}\} = FV(p)$.
- (b) $q \approx t[V] = \mathbf{match} \overline{V_l/x_l^l}$ iff $q[\overline{V_l/x_l^l}] = t[V]$, and $\{\bar{x}_l^l\} = FV(q)$.
- (c) $\langle \mathbf{self} A ; \bar{r}^n, r \rangle \approx \langle V ; \bar{s}^n \rangle = \mathbf{partial} \overline{V_l/x_l^l}$ iff $\langle \mathbf{self} A ; \bar{r}^n \rangle \approx \langle V ; \bar{s}^n \rangle = \mathbf{match} \overline{V_l/x_l^l}$.

PROOF. (a) PROOF. ($\rightarrow$) By induction on p :

- ($p = x'$) Assuming $p \triangleright V' = \mathbf{match} \overline{V/x}$, we must show that $x'[\overline{V/x}] = V'$, and that $\{\bar{x}\} = FV(x')$. We only have a single possible $\triangleright$ -case capable of generating a match with a variable pattern. According to this rule, we can infer $\overline{V/x} = V'/x'$. For the first goal, we follow by computation: $x'V'/x' = V'$. Lastly, $\{x'\} = FV(x')$.
- ($p = l$) Similarly, we only have one option for the inference rule. So, from our assumption, we can infer that: $V' = l'$, $l = l'$, and $\overline{V/x} = /$. Thus, we have that $l[] = l = l'$ and $\emptyset = FV(l)$.
- ($p = (\overline{a_i : p_i^i})$) From our assumption, we know $\forall i. p_i \triangleright V_i = \mathbf{match} \sigma_i$. From our induction hypothesis we know that σ_i transforms p_i in V_i . So, $(\overline{a_i : p_i^i})[\bar{\sigma}_i] = \{\overline{a_i : V_i^i}\}$. The induction hypothesis assures that a σ_i is a substitution of the free vars in p_i , and by definition $FV((\overline{a_i : p_i^i}))$ is the union of all sub-patterns free variables, thus our constraint is fulfilled.
- ($p = \{\bar{a} : \bar{p}, \dots x\}$) Our $\triangleright$ hypothesis can only be constructed from a single rule, which provides us that $\forall i. p_i \triangleright V_i = \mathbf{match} \sigma_i$. By our IH, we can fill up the holes to obtain V_i from of any p_i using σ_i . Again, from our IH, it follows that the variables being substituted in a σ_i are exactly the free variables of p_i . The free variables of $\{\overline{a_i : p_i^i}, \dots x\}$ are the union of the free variables of all p_i and x , so we can conclude that all substitutions in $\bar{\sigma}_i^i, \{\overline{a_j : V_j^j}\}/x$ span the free variables of $\{\overline{a_i : p_i^i}, \dots x\}$. And $\{\overline{a_i : p_i^i}, \dots x\}[\bar{\sigma}_i^i, \{\overline{a_j : V_j^j}\}/x] = \{\overline{a_i : V_i^i}, \overline{a_j : V_j^j}\}$ follows from the definition of substitution on row patterns.

($\leftarrow$) By induction on p :

- ($p = x'$) A variable pattern is always satisfied, so $x \triangleright V' = \mathbf{match} \overline{V'/x}$.
- ($p = l$) Because $FV(l) = \emptyset$, we know that $l[] = l = V'$. Therefore, we can conclude $l \triangleright V' = \mathbf{match} /$.
- ($p = (\overline{a_i : p_i^i})$) By the definition of substitution, $\{\bar{x}\} = FV(\overline{a_i : p_i^i}) = \bigcup_i FV(p_i)$. Therefore, we can partition the substitutions in $\overline{V/x}$ into $\overline{V_{p_i}/x_{p_i}^{p_i}}$. From our assumption, $V' = (\overline{a_i : p_i^i})[\overline{V_{p_i}/x_{p_i}^{p_i}}]$, so $V' = \{\overline{a_i : V_i^i}\}$ with $V_i = p_i[\overline{V_{p_i}/x_{p_i}^{p_i}}]$. By our induction hypothesis, we can conclude that $p_i \triangleright V_i = \mathbf{match} \overline{V_{p_i}/x_{p_i}^{p_i}}$ for every i . Therefore, we can conclude $(\overline{a_i : p_i^i}) \triangleright \{\overline{a_i : V_i^i}\} = \mathbf{match} \overline{V_{p_i}/x_{p_i}^{p_i}}$.
- ($p = \{\overline{a_i : p_i^i}, \dots y\}$) The free variables of a row pattern are: $FV(\{\overline{a_i : p_i^i}, \dots y\}) = \bigcup_i FV(p_i) \cup FV(y)$. From our assumption, we can give a more specific shape for $V' = \{\overline{a_i : p_i^i}, \dots y\}[\overline{V_{p_i}/x_{p_i}^{p_i}}, V_y/y]$. According to the definition of substitution on row patterns, we can infer that $V' = \{\overline{a_i : V_i^i}, \overline{a_j : V_j^j}\}$, with $V_i = p_i[\overline{V_{p_i}/x_{p_i}^{p_i}}]$ and $V_y = \{\overline{a_j : V_j^j}\}$. By our induction hypothesis, we can conclude that $p_i \triangleright V_i = \mathbf{match} \overline{V_{p_i}/x_{p_i}^{p_i}}$

for every i . Therefore, we can use the row pattern match rule to conclude $\{\overline{a_i : p_i^i}, \dots, y\} \triangleright \{\overline{a_i : V_i^i}, \overline{a_j : V_j^j}\} = \mathbf{match} [\overline{V_{p_i} / x_{p_i}^{p_i^i}}, \{\overline{a_j : V_j^j}\} / y]$.

□

(b) PROOF. Let $q = \langle \mathbf{self} A ; \bar{r}^n \rangle$ and $t[V] = \langle V ; \bar{s}^n \rangle$.

($\rightarrow$) Assume $q \approx t[V] = \mathbf{match} \overline{V_l / x_l^l}$. The only applicable rule is the full-match rule (Rule 1), whose premises give us $A \triangleright V = \mathbf{match} \sigma$ and $\forall k \leq n. r_k \smile s_k = \mathbf{match} \sigma_k$, where $\overline{V_l / x_l^l} = \sigma, \overline{\sigma_k^k}$. By lemma 4.1(a), $A[\sigma] = V$. For each step, inspecting the $\smile$ rules:

- If $r_k = \square.a$, then $s_k = \square.a$ and $\sigma_k = /$, so $r_k[\sigma_k] = \square.a = s_k$.
- If $r_k = \square A'$, then $s_k = \square(V')$ with $A' \triangleright V' = \mathbf{match} \sigma_k$. By lemma 4.1(a), $A'[\sigma_k] = V'$, so $r_k[\sigma_k] = \square V' = s_k$.

Hence $q[\overline{V_l / x_l^l}] = \langle V ; \bar{s}^n \rangle = t[V]$. Since copatterns have no repeated free variables, the domains of $\sigma, \overline{\sigma_k^k}$ partition $FV(q) = FV(A) \cup \bigcup_k FV(r_k)$, so $\{\overline{x_l^l}\} = FV(q)$.

($\leftarrow$) Assume $q[\overline{V_l / x_l^l}] = t[V]$ and $\{\overline{x_l^l}\} = FV(q)$. Unfolding, $A[\sigma] = V$ and $r_k[\sigma_k] = s_k$ for each $k \leq n$, where $\sigma = \overline{V_l / x_l^l} \upharpoonright_{FV(A)}$ and $\sigma_k = \overline{V_l / x_l^l} \upharpoonright_{FV(r_k)}$. Since copatterns have no repeated free variables, $FV(A), FV(r_1), \dots, FV(r_n)$ are pairwise disjoint and together equal $FV(q) = \{\overline{x_l^l}\}$, so the restrictions are well-defined. By lemma 4.1(a), $A \triangleright V = \mathbf{match} \sigma$. For each step, the $\smile$ rules give $r_k \smile s_k = \mathbf{match} \sigma_k$:

- $r_k = \square.a$ and $s_k = \square.a$: the attribute $\smile$ rule applies directly.
- $r_k = \square A'$ and $s_k = \square(V')$ with $A'[\sigma_k] = V'$: by lemma 4.1(a), $A' \triangleright V' = \mathbf{match} \sigma_k$, so the call $\smile$ rule applies.

Since both sides carry the same index n , the full-match rule (Rule 1) applies, giving $q \approx t[V] = \mathbf{match} \overline{V_l / x_l^l}$.

□

(c) PROOF. ($\rightarrow$) Assume $\langle \mathbf{self} A ; \bar{r}^n, r \rangle \approx \langle V ; \bar{s}^n \rangle = \mathbf{partial} \overline{V_l / x_l^l}$. The partial-match rule (Rule 2) fires precisely when the setting is shorter than the copattern; its premises are the same as Rule 1 applied to the first n steps: $A \triangleright V = \mathbf{match} \sigma$ and $\forall k \leq n. r_k \smile s_k = \mathbf{match} \sigma_k$. Those premises are exactly those of the full-match rule applied to $\langle \mathbf{self} A ; \bar{r}^n \rangle$ against $\langle V ; \bar{s}^n \rangle$, giving $\langle \mathbf{self} A ; \bar{r}^n \rangle \approx \langle V ; \bar{s}^n \rangle = \mathbf{match} \overline{V_l / x_l^l}$.

($\leftarrow$) Assume $\langle \mathbf{self} A ; \bar{r}^n \rangle \approx \langle V ; \bar{s}^n \rangle = \mathbf{match} \overline{V_l / x_l^l}$. By Rule 1, $A \triangleright V = \mathbf{match} \sigma$ and $\forall k \leq n. r_k \smile s_k = \mathbf{match} \sigma_k$. For $\langle \mathbf{self} A ; \bar{r}^n, r \rangle$ against $\langle V ; \bar{s}^n \rangle$, the partial-match rule applies since $n+1 > n$ and the first n steps already match. Hence $\langle \mathbf{self} A ; \bar{r}^n, r \rangle \approx \langle V ; \bar{s}^n \rangle = \mathbf{partial} \overline{V_l / x_l^l}$.

□

□

LEMMA B.1 (DETERMINISM). (a) For all p and closed V , exactly one of the following holds: $p \triangleright V = \mathbf{match} \sigma$ for some substitution σ , or $p \triangleright V = \mathbf{fail}$.

(b) For all q and closed $t[V]$, exactly one of the following holds: $q \approx t[V] = \mathbf{match} \sigma$ for some σ , $q \approx t[V] = \mathbf{partial} \sigma$ for some σ , or $q \approx t[V] = \mathbf{fail}$.

PROOF. (a) By induction on p and tracing which inference rule is compatible:

- ($p = x$), a variable is always fulfilled, so this is always a $\mathbf{match} [V/x]$.
- ($p = n$), let us consider each case for V .
 - ($V = n'$), if $n = n'$ then $\mathbf{match}$ else $\mathbf{fail}$.
 - Otherwise, it is a $\mathbf{fail}$.
- ($p = (\overline{a_i : p_i^i})$), let us consider each case for V .

- $(V = \{\overline{b_j : V_j^j}\})$ If the label sets match $(\overline{a_i^i} = \overline{b_j^j})$ as sets, with $i = j$, we apply the IH to each p_i against V_i . If every $p_i \triangleright V_i = \mathbf{match} \sigma_i$, the strict-dict match rule gives $\mathbf{match} \overline{\sigma_i}$. If some $p_i \triangleright V_i = \mathbf{fail}$, the sub-pattern failure rule gives $\mathbf{fail}$. By the IH exactly one outcome holds for each sub-pattern, so exactly one overall outcome holds. If the label sets differ $(\overline{a_i^i} \neq \overline{b_j^j})$, the label-mismatch rule gives $\mathbf{fail}$.
 - $(V \neq \{\overline{b_j : V_j^j}\})$ The last $\triangleright$ rule for strict patterns gives $\mathbf{fail}$.
 - $(p = \{\overline{a_i : p_i^i}, \dots, x\})$, let us consider each case for V .
 - $(V = \{\overline{a_i : V_i^i}, \overline{b_j : W_j^j}\})$ where $\overline{a}$ are exactly the required keys and $\overline{b}$ are the extra keys. We apply the IH to each p_i against V_i . If every $p_i \triangleright V_i = \mathbf{match} \sigma_i$, the row-dict match rule gives $\mathbf{match} \overline{\sigma_i}, \{\overline{b_j : W_j^j}\}/x$. If some $p_i \triangleright V_i = \mathbf{fail}$, the row sub-pattern failure rule gives $\mathbf{fail}$. By the IH exactly one outcome holds for each sub-pattern, so exactly one overall outcome holds.
 - $(V$ does not contain all required keys $\overline{a}$, or V is not a dictionary) The last $\triangleright$ rule for row patterns gives $\mathbf{fail}$.
- (b) Since q has the single form $\langle \mathbf{self} A ; \overline{r^n} \rangle$ and $t[V] = \langle V ; \overline{s^n} \rangle$, the case split is on which of the five $\approx$ rules applies. The rules are mutually exclusive by their premises (they test disjoint conditions on the lengths and the root/step matches), so exactly one fires:
- (Rule 1, **match**): requires $n = m$, $A \triangleright \{\overline{l : V}\} = \mathbf{match} \sigma$, and $\forall k \leq n. r_k \smile s_k = \mathbf{match} \sigma_k$. By part (a) of this lemma, $A \triangleright V$ is either **match** or **fail**, and by determinism of $\smile$ (each step-pair has at most one applicable rule), all premises are decided. If all hold, exactly this rule gives $\mathbf{match} \sigma, \overline{\sigma_k^k}$.
 - (Rule 2, **partial**): requires $n > m$ and the same root/step premises for the first m steps. Disjoint from Rule 1 by the length condition $n > m$.
 - (Rule 3, **fail**): requires $n < m$. Disjoint from Rules 1 and 2.
 - (Rule 4, **fail**): requires the root to match but some step k to fail. Disjoint from Rules 1 and 2 by the existence of a failing step.
 - (Rule 5, **fail**): requires $A \triangleright V = \mathbf{fail}$. Disjoint from Rules 1–4 which all require the root to match.
- These five cases cover all combinations of length comparison and root/step outcomes, so exactly one of **match**, **partial**, or **fail** holds. □

B.2 (Co)Pattern matching is preserved by $\llbracket _ \rrbracket$

LEMMA B.2 (EQ-EQB). *For literals $l, l' : l = l' \text{ iff } l == l' \rightarrow^* \mathbf{True}$.*

PROOF. By the Fw_{p_y} rules (eq_t) and (eq_f) . If $l = l'$ then (eq_t) gives $l == l' \mapsto \mathbf{True}$. If $l \neq l'$ then (eq_f) gives $l == l' \mapsto \mathbf{False}$, so $l == l' \not\mapsto^* \mathbf{True}$. Since $\llbracket l \rrbracket = l$ by Lemma B.8, the same holds for the translated literals. □

LEMMA B.3 (NEQ-NEQB). *For literals $l, l' : l \neq l' \text{ iff } l == l' \rightarrow^* \mathbf{False}$.*

PROOF. By the Fw_{p_y} rules (eq_t) and (eq_f) . If $l \neq l'$ then (eq_f) gives $l == l' \mapsto \mathbf{False}$. If $l = l'$ then (eq_t) gives $l == l' \mapsto \mathbf{True}$, so $l == l' \not\mapsto^* \mathbf{False}$. Since $\llbracket l \rrbracket = l$ by Lemma B.8, the same holds for the translated literals. □

LEMMA B.4 (STRENGTHEN E). *If $BV(D) \not\subseteq FV(e)$ then $D; e = e$ (observationally).*

PROOF. Let $c = BV(D)$ (either a class name or an instance variable). Since $c \notin FV(e)$, no subterm of e mentions c . The Fw_{py} rules that consume a definition are (*new*), (*attr*), (*meth*), (*call*), and (*else*): each requires c to appear free in the redex. Since $c \notin FV(e)$, none of these rules can fire using D 's binding during any reduction of e ; D is permanently inert.

To see observational equivalence, let $C[\cdot]$ be any closing context. We argue $C[D; e]$ and $C[e]$ produce the same observable results. Every reduction step of $C[D; e]$ either:

- takes place entirely inside e (or the context C), with D acting as a transparent prefix — the same step is available in $C[e]$; or
- is an application of the (*def*) rule, which bubbles D outward past a frame: $F[D; V] \rightarrow D; F[V]$. Since $c \notin FV(e)$ and therefore c never appears in any value produced by evaluating e , the (*def*) bubble steps only rearrange where D sits in the context and do not affect the observable result.

In either case the reduction of $C[e]$ mirrors that of $C[D; e]$ step-for-step (ignoring the inert prefix), yielding the same final value. Hence $D; e = e$ observationally. $\square$

LEMMA 5.2 ($\llbracket _ \rrbracket$ PRESERVES PATTERN MATCHING). (a) $p \triangleright V = \text{match } \overline{V/x} \text{ iff } \llbracket p \rrbracket.\text{match}(\llbracket V \rrbracket) \rightarrow_* \{x : \llbracket V \rrbracket\}$.
 (b) $p \triangleright V = \text{fail}$ iff $\llbracket p \rrbracket.\text{match}(\llbracket V \rrbracket) \rightarrow_* \text{None}$.

(a) PROOF. ($\rightarrow$) By induction on p :

- ($p = x$) Thus we know: $x \triangleright V = \text{match } V/x$ and that our goal is $\{x : \llbracket V \rrbracket\}$. Let's follow by computation:

$$\begin{aligned}
 & \llbracket x \rrbracket.\text{match}(\llbracket V \rrbracket) \\
 &= PVar(\text{name} = 'x').\text{match}(\llbracket V \rrbracket) \\
 &= \text{new } x_c := PVar(\text{name} = 'x'); x_c.\text{match}(\llbracket V \rrbracket) && (\text{new}) \\
 &= \text{new } x_c := PVar(\text{name} = 'x'); \{\text{self.name} : v\} [x_c/\text{self}, \llbracket V \rrbracket/v] && (\text{meth}) \\
 &= \text{new } x_c := PVar(\text{name} = 'x'); \{x_c.\text{name} : \llbracket V \rrbracket\} \\
 &= \text{new } x_c := PVar(\text{name} = 'x'); \{x : \llbracket V \rrbracket\} && (\text{attr}) \\
 &= \{x : \llbracket V \rrbracket\} && (\text{lemma B.4})
 \end{aligned}$$

- ($p = l$) Thus we know $l \triangleright l' = \text{match } [/], l = l'$, and that our goal is $\{\}$. Let's follow by computation:

$$\begin{aligned}
 \llbracket l = l' \rrbracket.\text{match}(\llbracket l' \rrbracket) &= PLit(l = l').\text{match}(l') \\
 &= \text{new } x := PLit(l = l'); x.\text{match}(l') && (\text{new}) \\
 &= (\text{if self.l} == v \text{ then } \{\} \text{ else None}) [x/\text{self}, \llbracket l' \rrbracket/v] && (\text{meth}) \\
 &= (\text{if } x.l == l' \text{ then } \{\} \text{ else None}) \\
 &= \text{if } l == l' \text{ then } \{\} \text{ else None} && (\text{attr}) \\
 &= \text{if True then } \{\} \text{ else None} && (\text{lemma B.2}) \\
 &= \{\} && (\text{if})
 \end{aligned}$$

- ($p = (\overline{l_i : p_i})^i$) Thus we know $\forall i. p_i \triangleright V_i = \text{match } \sigma_i$ and that our goal is $\{\overline{l_{p_i} : \llbracket V_{p_i} \rrbracket^{p_i}}^i\}$. We can use our IH together with our assumptions to conclude that $\llbracket p_i \rrbracket.\text{match}(\llbracket V_i \rrbracket)$ is $\{\overline{l_{p_i} : V_{p_i}^{p_i}}\}$ for every i . The body of the match method of a *PDict* pattern is a fold that

checks if the patterns in the row match the corresponding entries in the value, and if so, it accumulates the results of those matches in a dictionary. We know that both p and V have the same labels, so we know that the fold will check every pattern in p against the corresponding entry in V , and by our assumptions and IH, we know that all those checks will succeed and accumulate the results in a dictionary of the form $\overline{\overline{\{l_{p_i} : V_{p_i}^{p_i}\}}}$.

- ($p = \overline{\{l_i : p_i, \dots x\}}$) Then we know we can divide the in V into two parts, one that matches the patterns p_i and another part that is the rest of the dictionary.

$$\begin{aligned}
 & \llbracket \overline{\{l_i : p_i, \dots x\}} \rrbracket . \text{match}(\llbracket \overline{\{l_i : V_i, l_j : V_j\}} \rrbracket) \\
 & \quad (\text{where } \llbracket \overline{\{l_i : p_i, \dots x\}} \rrbracket = \text{PRow}(pats = \overline{\{l_i : \llbracket p_i \rrbracket\}}, r = 'x')) \\
 & = X; x. \text{match}(\llbracket \overline{\{l_i : V_i, l_j : V_j\}} \rrbracket) \quad (\text{new}) \\
 & (\text{where } X = \text{new } x_o := \text{PRow}(pats = \overline{\{l_i : \llbracket p_i \rrbracket\}}, r = 'x')) \\
 & = \dots \quad (\text{meth})
 \end{aligned}$$

With *meth*, we call the match method and easily pass through the first check since $\llbracket V \rrbracket$ is dictionary. Thus, we are left with a fold checking if the matching entry on V' satisfies the pattern. After the substitutions, we have the following term:

$$\text{fold}(f1, o_p, \{x : v - o_p\})$$

Where:

$$\begin{aligned}
 o_p &= \overline{\{l_i : \llbracket p_i \rrbracket\}} \\
 v &= \llbracket \overline{\{l_i : V_i, l_j : V_j\}} \rrbracket \\
 v - o_p &= \overline{\{l_j : \llbracket V_j \rrbracket\}} \\
 f1 &= \lambda acc, k : \text{if } acc == \text{None then None else } f2(\text{self}.pats[k], acc, k) \\
 f2 &= \lambda p, acc, k : \text{let } r := p.\text{match}(v) \text{ in if } r == \text{None then None else } acc ++ res
 \end{aligned}$$

We know that we will always have at least one entry in the dictionary being used as the initial value of the fold, so we know we won't start with fold_1 . Thus, the fold will be translated into a sequence of telescoped applications of f_1 with the result of the previous application as the accumulator, and the next literal and value of o_p . From our assumption $\forall i. p_i \triangleright V_i = \text{match } \overline{V_{j_i}/l_{j_i}^{j_i}}$, we can use our induction hypothesis to conclude that $\llbracket p_i \rrbracket . \text{match}(\llbracket V_i \rrbracket) \rightarrow_* \overline{\{l_{j_i} : V_{j_i}^{j_i}\}}$ for every i . This suffices to use our induction hypothesis on every application of f_1 . The helper f_2 will not ever receive a pattern that fails (according to our assumptions), so for every application of f_2 we will have that p is a pattern that matches the value v , thus we can use our induction hypothesis to conclude that $p.\text{match}(v) \rightarrow_* \overline{\{l_{j_i} : V_{j_i}^{j_i}\}}$. This is enough to conclude

that the whole fold will evaluate to $\overline{\{l_{j_i} : V_{j_i}^{j_i}, x : \overline{\{l_j : \llbracket V_j \rrbracket\}}\}}$

($\leftarrow$) Induction on p :

- ($p = x$) A variable pattern is always satisfied, so $x \triangleright V = \text{match } V/x$.

- $(p = l)$ Since we have a non-None result, we know that $l == V = \text{True}$, thus $l = V$ by lemma B.2. Therefore, $l \triangleright l = \text{match } /$.
- $(p = \overline{(l_i : p_i)}^i)$ According to our assumption, by analysis we discover that V is a dictionary with the same labels as p , and that for every pattern p_i and corresponding value V_i we have that $\llbracket p_i \rrbracket.\text{match}(\llbracket V_i \rrbracket) \rightarrow^* \{\overline{l_{j_i} : V_{j_i}}^{j_i}\}$. We can use our induction hypothesis to conclude that $p_i \triangleright V_i = \text{match } [\overline{V_{j_i}/l_{j_i}}^{j_i}]$ for every i . Therefore, we can conclude that $\overline{(l_i : p_i)}^i \triangleright \{\overline{l_i : V_i}^i\} = \text{match } \overline{V_{j_i}/l_{j_i}}^{j_i}$.
- $(p = \overline{\{l_i : p_i, \dots x\}}^i)$ Using a similar argument, but we have fewer constraints. We know that V has at least the same keys as p . And by our IH, $p_i \triangleright V_i = \text{match } [\overline{V_{p_i}/l_{p_i}}^{p_i}]$ for every i . The body of $P\text{Row}.\text{match}$ starts with $\text{self}.r : \text{diff}(v, \text{self}.pats)$ as the fold's accumulator. This dictionary is akin to the “rest” of the dictionary, so we can use it to conclude that $\overline{\{l_i : p_i, \dots x\}}^i \triangleright \{\overline{l_i : V_i}^i, \overline{l_j : V_j}^j\} = \text{match } \overline{V_{p_i}/l_{p_i}}^{p_i}, \{\overline{l_j : V_j}^j\} / x$.

□

(b) PROOF. ($\rightarrow$) By induction on p :

- $(p = x)$ Vacuous, since a variable pattern is always satisfied.
- $(p = l)$ The only way to fail a literal pattern is if the value is not equal to the literal. Thus, $l \neq V$.

$$\begin{aligned}
\llbracket l \rrbracket.\text{match}(\llbracket V \rrbracket) &= P\text{Lit}(l).\text{match}(\llbracket V \rrbracket) \\
&= \text{new } x := P\text{Lit}(l); x.\text{match}(\llbracket V \rrbracket) && (\text{new}) \\
&= (\text{if } \text{self}.l == v \text{ then } \{\} \text{ else None})[x/\text{self}, \llbracket V \rrbracket/v] && (\text{meth}) \\
&= \text{if } x.l == \llbracket V \rrbracket \text{ then } \{\} \text{ else None} \\
&= \text{if } l == \llbracket V \rrbracket \text{ then } \{\} \text{ else None} && (\text{attr}) \\
&= \text{None} && (\text{lemma B.3})
\end{aligned}$$

- $(p = \overline{(l_i : p_i)}^i)$ By assumption $p \triangleright V = \text{fail}$. We trace which $\triangleright$ failure rule applies.
 - $(V \neq \{\overline{b_j : V_j}^j\})$ $P\text{Dict}.\text{match}$ guards with $\neg \text{isinstance}(v, \text{dict})$ and returns **None** immediately.
 - (Label-set mismatch, $\overline{l_i}^i \neq \overline{b_j}^j$) $P\text{Dict}.\text{match}$ guards with $\text{keys}(v) \neq \text{keys}(\text{self}.pats)$ and returns **None** immediately.
 - $(\exists i. p_i \triangleright V_i = \text{fail})$ By the induction hypothesis, $\llbracket p_i \rrbracket.\text{match}(\llbracket V_i \rrbracket) \rightarrow^* \text{None}$. Inside the fold, the step for index i yields **None**, which cascades through all remaining fold steps via the $\text{sub} = \text{None}$ guard, so the whole fold evaluates to **None**.
- $(p = \overline{\{l_i : p_i, \dots x\}}^i)$ By assumption $p \triangleright V = \text{fail}$. We trace which $\triangleright$ failure rule applies.
 - $(V \neq \{\overline{b_j : V_j}^j\})$ The key lookup $v[k]$ or the difference $v - \text{self}.pats$ is undefined for a non-dictionary V , so $P\text{Row}.\text{match}$ returns **None**.
 - $(\exists i. p_i \triangleright V_i = \text{fail})$ By the induction hypothesis, $\llbracket p_i \rrbracket.\text{match}(\llbracket V_i \rrbracket) \rightarrow^* \text{None}$. As in the $P\text{Dict}$ case, this **None** cascades through the fold via the $\text{sub} = \text{None}$ guard, so the whole fold evaluates to **None**.

($\leftarrow$) By induction on p :

- $(p = x)$ Vacuous: $P\text{Var}.\text{match}(\text{self}, v) = \{\text{self}.name : v\}$ always returns a non-None dict, so $\llbracket x \rrbracket.\text{match}(\llbracket V \rrbracket) \rightarrow^* \text{None}$ is impossible and the implication holds vacuously.
- $(p = l)$ $\llbracket l \rrbracket.\text{match}(\llbracket V \rrbracket) \rightarrow^* \text{None}$ iff the $P\text{Lit}.\text{match}$ body takes the else branch, i.e. $\text{self}.l \neq v$, i.e. $l \neq V$. By Lemma B.3, $l \neq V$, so the literal failure rule gives $l \triangleright V = \text{fail}$.

- $(p = (\overline{l_i : p_i^i})) \llbracket p \rrbracket . \text{match}(\llbracket V \rrbracket) \rightarrow^* \text{None}$ arises from three sub-cases in *PDict.match*:
 - V is not a dictionary: the $\neg \text{instance}(v, \text{dict})$ guard fires, returning **None**. The last $\triangleright$ rule for strict patterns gives $p \triangleright V = \text{fail}$.
 - $\text{keys}(\llbracket V \rrbracket) \neq \text{keys}(\text{self.pats})$: label sets differ, so the key-equality guard returns **None**. The strict-dict label-mismatch failure rule gives $p \triangleright V = \text{fail}$.
 - The fold returns **None**: there exists some k such that $\text{self.pats}[k].\text{match}(\llbracket V \rrbracket[k]) \rightarrow^* \text{None}$. By the induction hypothesis, $p_k \triangleright V_k = \text{fail}$. The sub-pattern failure rule gives $(\overline{l_i : p_i^i}) \triangleright \{l_i : V_i^i\} = \text{fail}$.
- $(p = (\overline{l_i : p_i^i}, \dots x)) \llbracket p \rrbracket . \text{match}(\llbracket V \rrbracket) \rightarrow^* \text{None}$ arises from two sub-cases in *PRow.match*:
 - V is not a dictionary or does not contain all required keys: the computation $v - \text{self.pats}$ is undefined or key lookup $v[k]$ fails, yielding **None**. The last $\triangleright$ rule for row patterns ($V \neq \{b_j : V_j^j\}$) gives $p \triangleright V = \text{fail}$.
 - The fold returns **None**: there exists some k such that $\text{self.pats}[k].\text{match}(\llbracket V \rrbracket[k]) \rightarrow^* \text{None}$. By the induction hypothesis, $p_k \triangleright V_k = \text{fail}$. The row sub-pattern failure rule gives $\{l_i : p_i^i, \dots x\} \triangleright V = \text{fail}$.

□

LEMMA 5.3 ($\llbracket _ \rrbracket$ PRESERVES COPATTERN MATCHING). Let bb be a co-object with $bb.st = \llbracket \langle V ; \bar{s} \rangle \rrbracket$.

- (a) $q \approx \langle V ; \bar{s} \rangle = \text{match } \overline{V/x} \text{ or } q \approx \langle V ; \bar{s} \rangle = \text{partial } \overline{V/x}$ iff $\llbracket q \rrbracket . \text{comatch}(bb) \rightarrow^* \{x : \llbracket V \rrbracket\}$.
 (b) $q \approx \langle V ; \bar{s} \rangle = \text{fail}$ iff $\llbracket q \rrbracket . \text{comatch}(bb) \rightarrow^* \text{None}$.

Let $q = \langle \text{self } A ; \bar{r}^n \rangle$ and $bb.st = \llbracket \langle V ; \bar{s} \rangle \rrbracket = [T\text{Self}(\llbracket V \rrbracket), \llbracket s_1 \rrbracket, \dots, \llbracket s_m \rrbracket]$. The translation gives $\llbracket q \rrbracket = [Q\text{Self}(\llbracket A \rrbracket), \llbracket r_1 \rrbracket, \dots, \llbracket r_n \rrbracket]$ stored as $o.q$ inside the *Obs* object $o = \llbracket (q = e) \rrbracket$. The *Obs.comatch*(o, bb) folds a step-checker f over the shorter of $o.q$ and $bb.st$:

$f(\text{sub}, k) = \text{let } r = o.q[k].\text{comatch}(bb.st[k]) \text{ in if sub = None } \vee r = \text{None then None else sub ++ } r$

For each index k , the step-checker computes:

- $k = 0$: $Q\text{Self}(\llbracket A \rrbracket). \text{comatch}(T\text{Self}(\llbracket V \rrbracket)) = \llbracket A \rrbracket . \text{match}(\llbracket V \rrbracket)$. By lemma 5.2(a), this returns $\{x : \llbracket V_x \rrbracket\}$ iff $A \triangleright V = \text{match } \sigma$.
- $k > 0, r_k = \square, s_k = \square, a$: $Q\text{Attr}(a). \text{comatch}(T\text{Attr}(a)) = \{\}$ (empty bindings).
- $k > 0, r_k = \square, A', s_k = \square(V')$: $Q\text{App}(\llbracket A' \rrbracket). \text{comatch}(T\text{App}(\llbracket V' \rrbracket)) = \llbracket A' \rrbracket . \text{match}(\llbracket V' \rrbracket)$. By lemma 5.2(a), this returns $\{x' : \llbracket V'_x \rrbracket\}$ iff $A' \triangleright V' = \text{match } \sigma_k$.
- $k > 0$, step type mismatch: the relevant *QAttr* or *QApp* branch returns **None**.

The fold accumulates all per-step binding dicts, yielding $\{x : \llbracket V \rrbracket\}$ when all steps succeed.

(a) PROOF. ($\rightarrow$) We handle each case separately.

Match ($q \approx \langle V ; \bar{s}^n \rangle = \text{match } \sigma, \overline{\sigma_k^k}$). Rule 1 gives $n = m, A \triangleright V = \text{match } \sigma$, and $\forall k \leq n. r_k \sim s_k = \text{match } \sigma_k$. Since $\text{len}(o.q) = n+1 = m+1 = \text{len}(bb.st)$, the full-match branch of *Obs.comatch* fires, folding f over $o.q$. At $k = 0$: $\llbracket A \rrbracket . \text{match}(\llbracket V \rrbracket) \rightarrow^* \{x : \llbracket V_x \rrbracket\}$ by lemma 5.2(a). At each $k > 0$: since $r_k \sim s_k = \text{match } \sigma_k$, the step is compatible and the checker returns the bindings σ_k . The fold succeeds and accumulates $\sigma, \overline{\sigma_k^k} = \overline{V_l/x_l^l}$, giving $\{x_l : \llbracket V_l \rrbracket\}$.

Partial ($q \approx \langle V ; \bar{s}^m \rangle = \text{partial } \sigma, \overline{\sigma_k^k}$). Rule 2 gives $n > m, A \triangleright V = \text{match } \sigma$, and $\forall k \leq m. r_k \sim s_k = \text{match } \sigma_k$. Since $\text{len}(o.q) = n+1 > m+1 = \text{len}(bb.st)$, the partial-match branch fires, folding f over $bb.st$ ($m+1$ elements). By the same step-by-step argument as above, all $m+1$ checks succeed and the fold returns the partial binding dict $\{x_l : \llbracket V_l \rrbracket\}$.

($\leftarrow$) Assume $\llbracket q \rrbracket . \text{comatch}(bb) \rightarrow^* \{D\}$ for some non-**None** dict D .

- If $n+1 = m+1$: the full-match branch ran. The fold over $o.q$ returned non-None, so every step-check $f(sub, k)$ succeeded. At $k = 0$: $\llbracket A \rrbracket.match(\llbracket V \rrbracket) \not\rightarrow_* \text{None}$, so by lemma 5.2(a) backward, $A \triangleright V = \text{match } \sigma$. At each $k > 0$: the checker returned non-None, so either $r_k = \Box.a$ and $s_k = \Box.a$ (giving $\{\}$), or $r_k = \Box A'$, $s_k = \Box(V')$, and $\llbracket A' \rrbracket.match(\llbracket V' \rrbracket) \not\rightarrow_* \text{None}$, giving $r_k \sim s_k = \text{match } \sigma_k$ by lemma 5.2(a). By Rule 1, $q \approx \langle V ; \bar{s}^n \rangle = \text{match } \sigma, \bar{\sigma}_k^k$.
- If $n+1 > m+1$: the partial-match branch ran over $bb.st$. By the same reasoning for $k = 0, \dots, m$, all prefix steps match. By Rule 2, $q \approx \langle V ; \bar{s}^m \rangle = \text{partial } \sigma, \bar{\sigma}_k^k$.

□

(b) PROOF. $(\rightarrow)$ Assume $q \approx \langle V ; \bar{s} \rangle = \text{fail}$. We trace which Fail rule applies.

- (Rule 3, $n < m$): $len(o.q) = n+1 < m+1 = len(bb.st)$, so the **else None** branch of $Obs.comatch$ fires immediately.
- (Rule 4, step failure): $A \triangleright V = \text{match } \sigma$ and $\exists k \leq \min(n, m)$. $r_k \sim s_k = \text{fail}$. The fold runs until step k . At $k = 0$: $\llbracket A \rrbracket.match(\llbracket V \rrbracket) \not\rightarrow_* \text{None}$ (root matches), so $f(\{\}, 0) \neq \text{None}$. At the failing step k : $r_k \sim s_k = \text{fail}$ means either a type mismatch (returning None directly from $QAttr$ or $QApp$) or $A' \triangleright V' = \text{fail}$, whence $\llbracket A' \rrbracket.match(\llbracket V' \rrbracket) \rightarrow_* \text{None}$ by lemma 5.2(b). In both cases, $f(sub, k) = \text{None}$, and the fold propagates None for all remaining steps.
- (Rule 5, root failure): $A \triangleright V = \text{fail}$. At $k = 0$: $\llbracket A \rrbracket.match(\llbracket V \rrbracket) \rightarrow_* \text{None}$ by lemma 5.2(b), so $f(\{\}, 0) = \text{None}$ and the fold returns None.

In all cases, $\llbracket q \rrbracket.comatch(bb) \rightarrow_* \text{None}$.

$(\leftarrow)$ Assume $\llbracket q \rrbracket.comatch(bb) \rightarrow_* \text{None}$.

- If $n+1 < m+1$: the **else None** branch fired, so $n < m$, giving Rule 3.
- Otherwise, the fold returned None. Let k^* be the smallest index where $f(sub, k^*) = \text{None}$:
 - $k^* = 0$: $\llbracket A \rrbracket.match(\llbracket V \rrbracket) \rightarrow_* \text{None}$, so by lemma 5.2(b), $A \triangleright V = \text{fail}$, giving Rule 5.
 - $k^* > 0$: all steps before k^* succeeded, so in particular $A \triangleright V = \text{match } \sigma$ (from $k = 0$). At k^* :
 - * Type mismatch ($QAttr$ or $QApp$ sees wrong constructor): the step $r_{k^*} \sim s_{k^*} = \text{fail}$ by the mismatch rules.
 - * $QApp$ sees $TApp$ but $\llbracket A' \rrbracket.match(\llbracket V' \rrbracket) \rightarrow_* \text{None}$: by lemma 5.2(b), $A' \triangleright V' = \text{fail}$, so $r_{k^*} \sim s_{k^*} = \text{fail}$.

Either way, $\exists k \leq \min(n, m)$. $r_k \sim s_k = \text{fail}$, giving Rule 4.

□

B.3 Simulation

LEMMA B.5 (INST-EQ). If $\mathcal{X} = \text{new } x_1 := C_1(\overline{a_i : V_i^i})$, $\mathcal{Y} = \text{new } x_2 := C_2(\overline{a_i : V_i^i})$, $D_1 = \text{class } C_1 : I_1; \bar{M}$, and $D_2 = \text{class } C_2 : I_2; \bar{M}$, then $D_1; E_1[\mathcal{X}; E_2[x_1]] = D_2; E_1[\mathcal{Y}; E_2[x_2]]$.

PROOF. Here $=$ denotes observational equivalence. We show that every Fw_{p_y} rule that fires on x_1 (resp. x_2) produces the same result when x_1 is replaced by x_2 , so no closing context can distinguish the two. The relevant rules are those that look up either the class definition or the instance attribute dictionary.

- (*attr*) Accesses an entry from the instance attribute dictionary $\{\overline{a_i : V_i^i}\}$. Both $\mathcal{X}$ and $\mathcal{Y}$ store the same dictionary, so the result is identical.
- (*meth*) Looks up a method name m in the class method vector $\bar{M}$. Both D_1 and D_2 carry the same $\bar{M}$, so the method body and the substitution $\text{self} \mapsto x_k$ are identical.

- (*call*) Looks up the **call** method in $\overline{M}$. Same argument as (*meth*): both classes share $\overline{M}$.
- (*else*) Fires when the accessed name a is absent from both the attribute dictionary and the method vector $\overline{M}$. The absence condition and the **else** body depend only on $\overline{M}$ and the attribute dictionary, both of which are shared.
- (*def*) Bubbles a definition outward past a frame. This rule does not inspect the class or attributes of any particular instance; it only checks that the bound variable of the inner definition does not appear free in the frame. The same step is therefore available regardless of which class name (C_1 or C_2) is in scope.

The Fw_{py} syntax contains no construct for inspecting the class name of an instance (no *instance* or *type* expression), so C_1 and C_2 are the only potential source of difference and no rule exposes it. Hence, every reduction sequence starting from $D_1; E_1[\mathcal{X}; E_2[x_1]]$ has a corresponding sequence from $D_2; E_1[\mathcal{Y}; E_2[x_2]]$ producing the same observable result, establishing observational equivalence. $\square$

LEMMA B.6 (D-F). $\llbracket F[e] \rrbracket = \llbracket F \rrbracket[\llbracket e \rrbracket]$.

PROOF. By case analysis on the 14 frame forms from fig. 9.

Frames whose non-hole positions hold values. These are $V(\square)$, $V[\square]$, $V_1 + V_2 : \square$, $V_1 ? \square$, **fold**($V_1, V_2, \square$), $V_1 == \square$, and $\square.m$. In each case every non-hole sub-term is a value V , hence a Fw_{py} term. By Lemma B.8, $\llbracket V \rrbracket = V$, so $\llbracket F \rrbracket = F$. Then $\llbracket F[e] \rrbracket = F[\llbracket e \rrbracket] = \llbracket F \rrbracket[\llbracket e \rrbracket]$.

Frames with non-hole expression sub-terms. The translation acts on every sub-expression independently. We show two representative cases; the remaining ones are analogous.

- ($F = \square(e_2)$) $\llbracket F[e] \rrbracket = \llbracket \square(e_2) \rrbracket = \llbracket \square \rrbracket(\llbracket e_2 \rrbracket) = \llbracket F \rrbracket[\llbracket e \rrbracket]$, where $\llbracket F \rrbracket = \llbracket \square \rrbracket(\llbracket e_2 \rrbracket)$ by definition of $\llbracket _ \rrbracket$ on application.
- ($F = \text{if } \square \text{ then } e_2 \text{ else } e_3$) $\llbracket F[e] \rrbracket = \llbracket \text{if } e \text{ then } e_2 \text{ else } e_3 \rrbracket = \text{if } \llbracket e \rrbracket \text{ then } \llbracket e_2 \rrbracket \text{ else } \llbracket e_3 \rrbracket = \llbracket F \rrbracket[\llbracket e \rrbracket]$, where $\llbracket F \rrbracket = \text{if } \square \text{ then } \llbracket e_2 \rrbracket \text{ else } \llbracket e_3 \rrbracket$.
- ($F = \text{new } x := C(\square); e'$) $\llbracket F[e] \rrbracket = \llbracket \text{new } x := C(e); e' \rrbracket = \text{new } x := C(\llbracket e \rrbracket); \llbracket e' \rrbracket = \llbracket F \rrbracket[\llbracket e \rrbracket]$, where $\llbracket F \rrbracket = \text{new } x := C(\square); \llbracket e' \rrbracket$.

The remaining forms $\square[e_2]$, $\square + e_2 : e_3$, $V_1 + \square : e_3$, $\square ? e_2$, **fold**($\square, e_2, e_3$), **fold**($V_1, \square, e_3$), and $\square == e_2$ each follow by the same argument: the translation distributes over all sub-expressions, so the hole translates to $\llbracket e \rrbracket$ and every other position translates independently. $\square$

LEMMA B.7 (D-E). $\llbracket E[e] \rrbracket = \llbracket E \rrbracket[\llbracket e \rrbracket]$.

PROOF. By induction on E .

- ($E = \square$) $\llbracket \square[e] \rrbracket = \llbracket e \rrbracket = \llbracket \square \rrbracket[\llbracket e \rrbracket]$
- ($E = F[E']$) By lemma B.6: $\llbracket (F[E'])[e] \rrbracket = \llbracket F[E'[e]] \rrbracket = \llbracket F \rrbracket[\llbracket E'[e] \rrbracket]$. By IH, $\llbracket F \rrbracket[\llbracket E'[e] \rrbracket] = \llbracket F \rrbracket[\llbracket E' \rrbracket[\llbracket e \rrbracket]]$. By lemma B.6 applied to E' , $\llbracket F[E'] \rrbracket = \llbracket F \rrbracket[\llbracket E' \rrbracket]$, so $\llbracket F \rrbracket[\llbracket E' \rrbracket[\llbracket e \rrbracket]] = \llbracket F[E'] \rrbracket[\llbracket e \rrbracket]$.
- ($E = D; E'$) First, $\llbracket (D; E')[e] \rrbracket = \llbracket D; (E'[e]) \rrbracket$ by the definition of context plugging. Since the translation distributes over the definition prefix (as shown in the $D; e'$ case of Lemma B.8), $\llbracket D; (E'[e]) \rrbracket = \llbracket D \rrbracket; \llbracket E'[e] \rrbracket$. By IH on E' , $\llbracket E'[e] \rrbracket = \llbracket E' \rrbracket[\llbracket e \rrbracket]$, so $\llbracket D \rrbracket; \llbracket E'[e] \rrbracket = \llbracket D \rrbracket; (\llbracket E' \rrbracket[\llbracket e \rrbracket])$. Finally, applying the same distributivity in reverse, $\llbracket D \rrbracket; \llbracket E' \rrbracket = \llbracket D; E' \rrbracket$, so $\llbracket D \rrbracket; (\llbracket E' \rrbracket[\llbracket e \rrbracket]) = \llbracket D; E' \rrbracket[\llbracket e \rrbracket] = \llbracket E \rrbracket[\llbracket e \rrbracket]$.

$\square$

LEMMA B.8 (D-ID). If e is a Fw_{py} term, then $\llbracket e \rrbracket = e$.

PROOF. By structural induction on e . The translation $\llbracket _ \rrbracket$ distributes over every Fw_{p_y} syntactic constructor and defines non-identity cases only for the three new $Fw_{p_y}^{co}$ forms ($\mathbf{dec}$, $\llbracket \bar{O} \approx_0 t \rrbracket$, $q = e'$), none of which appear in a pure Fw_{p_y} term.

- $(e = l \text{ or } e = x)$ $\llbracket l \rrbracket = l$ and $\llbracket x \rrbracket = x$ directly.
- $(e = \lambda x. e')$ $\llbracket \lambda x. e' \rrbracket = \lambda x. \llbracket e' \rrbracket$; by IH $\llbracket e' \rrbracket = e'$, so $\llbracket \lambda x. e' \rrbracket = \lambda x. e'$.
- $(e = e_1(e_2))$ $\llbracket e_1(e_2) \rrbracket = \llbracket e_1 \rrbracket(\llbracket e_2 \rrbracket)$; by IH both translate to themselves, giving $e_1(e_2)$.
- $(e = \{l : e_i\}, e_1[e_2], e_1 + e_2 : e_3, e_1 ? e_2, \text{if } e_1 \text{ then } e_2 \text{ else } e_3, \mathbf{fold}(e_1, e_2, e_3), e_1 == e_2, e.m)$ Each is a congruence form: the translation distributes into each sub-expression, and by IH every sub-expression is left unchanged.
- $(e = D; e')$ $\llbracket D; e' \rrbracket = \llbracket D \rrbracket; \llbracket e' \rrbracket$; since D is a Fw_{p_y} definition, $\llbracket D \rrbracket = D$ (the Fw_{p_y} definition forms $\mathbf{class } C : I; \bar{M}$ and $\mathbf{new } x := C(V)$ carry no $Fw_{p_y}^{co}$ constructs), and by IH $\llbracket e' \rrbracket = e'$.
- $(e = \mathbf{new } x := C(e_1); e_2)$ $\llbracket \mathbf{new } x := C(e_1); e_2 \rrbracket = \mathbf{new } x := C(\llbracket e_1 \rrbracket); \llbracket e_2 \rrbracket$; by IH both sub-expressions translate to themselves.

□

LEMMA B.9 (SIM-DEC). *If $M \xrightarrow{dec} N$ for any $Fw_{p_y}^{co}$ terms, then there exists a Fw_{p_y} term T such that $\llbracket M \rrbracket \rightarrow^* T$ and $T = \llbracket N \rrbracket$ (modulo observational equivalence).*

PROOF. We have $M = D; E[C(V)]$, $N = D; E[\llbracket \bar{O} \approx_0 \langle V ; \rangle \rrbracket]$, $C \notin BV(E)$, and $D = \mathbf{dec } C : \bar{O}$. By lemma B.7:

$$\begin{aligned} \llbracket M \rrbracket &= \llbracket D \rrbracket; \llbracket E[C(V)] \rrbracket \\ \llbracket N \rrbracket &= \llbracket D \rrbracket; \llbracket E[CoData(\llbracket \bar{O} \rrbracket, \llbracket TSelf(\llbracket V \rrbracket), 0)] \rrbracket \end{aligned}$$

Each reduces in one Fw_{p_y} (new) step to a fresh allocation:

$$\begin{aligned} \llbracket M \rrbracket &\rightarrow_{new} \llbracket D \rrbracket; \mathcal{X}; \llbracket E \rrbracket[x_1] \\ \llbracket N \rrbracket &\rightarrow_{new} \llbracket D \rrbracket; \mathcal{Y}; \llbracket E \rrbracket[x_2] \end{aligned}$$

where $\mathcal{X} = \mathbf{new } x_1 := C(\{obvs : \llbracket \bar{O} \rrbracket, st : \llbracket TSelf(\llbracket V \rrbracket) \rrbracket, i : 0\})$ and $\mathcal{Y} = \mathbf{new } x_2 := CoData(\{obvs : \llbracket \bar{O} \rrbracket, st : \llbracket TSelf(\llbracket V \rrbracket) \rrbracket, i : 0\})$. Both instances share the same attribute dictionary; this relies on the translation setting $i = 0$ in C 's constructor. Since $\mathbf{class } C(CoData)$ inherits all methods $\bar{M}$ of $CoData$ without adding new ones, Lemma B.5 gives:

$$\llbracket D \rrbracket; \mathcal{X}; \llbracket E \rrbracket[x_1] = \llbracket D \rrbracket; \mathcal{Y}; \llbracket E \rrbracket[x_2]$$

Thus both $\llbracket M \rrbracket$ and $\llbracket N \rrbracket$ reduce in one step to observationally equivalent terms, establishing the simulation. □

LEMMA B.10 (D-SUBST). *For any $Fw_{p_y}^{co}$ expression e and substitution σ , $\llbracket e[\sigma] \rrbracket = \llbracket e \rrbracket[\llbracket \sigma \rrbracket]$, where $\llbracket \sigma \rrbracket = \{\llbracket V \rrbracket/x \mid V/x \in \sigma\}$.*

PROOF. By structural induction on e .

Base cases (fw terms).

- $(e = x, \text{ variable})$ If $x \in \text{dom}(\sigma)$, then $\llbracket x[\sigma] \rrbracket = \llbracket \sigma(x) \rrbracket$ and $\llbracket x \rrbracket[\llbracket \sigma \rrbracket] = x[\llbracket \sigma \rrbracket] = \llbracket \sigma(x) \rrbracket$. If $x \notin \text{dom}(\sigma)$, both sides equal x .
- $(e = l, \text{ literal})$ Substitution is the identity on literals, and $\llbracket l \rrbracket = l$ by Lemma B.8, so both sides equal l .
- $(e = \lambda x. e')$ Both sides compute $\lambda x. \llbracket e'[\sigma_{-x}] \rrbracket$ by Lemma B.8 and the IH, where σ_{-x} removes x from σ .

- $(e_1(e_2), \{\overline{l : e}\}, e_1[e_2], \text{if } e_1 \text{ then } e_2 \text{ else } e_3, \text{fold}(e_1, e_2, e_3), e_1 == e_2, e.m)$ Each is a congruence form: substitution distributes over all sub-expressions, and by the IH the translation commutes with each. Since all these are Fw_{py} forms, Lemma B.8 also gives $\llbracket e \rrbracket = e$, so the identity is immediate.

Binder cases (fw).

- $(e = \text{new } x := C(V); e')$ Substitution avoids the bound variable x , giving $\llbracket (\text{new } x := C(V); e')[\sigma] \rrbracket = \llbracket \text{new } x := C(V[\sigma]); e'[\sigma_{-x}] \rrbracket$. Since $\text{new } x := C(\cdot)$ is a Fw_{py} form, by Lemma B.8 and IH:

$$= \text{new } x := C(\llbracket V \rrbracket[\llbracket \sigma \rrbracket]); \llbracket e' \rrbracket[\llbracket \sigma_{-x} \rrbracket] = (\text{new } x := C(\llbracket V \rrbracket); \llbracket e' \rrbracket)[\llbracket \sigma \rrbracket] = \llbracket \text{new } x := C(V); e' \rrbracket[\llbracket \sigma \rrbracket].$$

New Fw_{py}^{co} cases.

- $(e = D; e' \text{ where } D = \text{dec } C : \overline{O})$ The bound name C is fresh w.r.t. σ (assuming standard variable hygiene). Substitution acts on each observation body and on e' :

$$\llbracket (\text{dec } C : \overline{O}; e')[\sigma] \rrbracket = \llbracket \text{dec } C : \overline{O}[\sigma]; e'[\sigma_{-C}] \rrbracket$$

By definition of $\llbracket _ \rrbracket$ on declarations and IH on each observation body e_i and on e' :

$$= \text{class } C(\text{CoData}) : \dots [\llbracket \overline{O}[\sigma] \rrbracket]; \llbracket e'[\sigma_{-C}] \rrbracket$$

$$= (\text{class } C(\text{CoData}) : \dots [\llbracket \overline{O} \rrbracket]; \llbracket e' \rrbracket)[\llbracket \sigma \rrbracket] = \llbracket \text{dec } C : \overline{O}; e' \rrbracket[\llbracket \sigma \rrbracket].$$

- $(e = \{\overline{O} \approx_i \langle V; \bar{s} \rangle\}, \text{co-object})$ Substitution distributes over the observation list, the root value, and the setting steps:

$$\begin{aligned} \llbracket \{\overline{O} \approx_i \langle V; \bar{s} \rangle\}[\sigma] \rrbracket &= \llbracket \{\overline{O}[\sigma] \approx_i \langle V[\sigma]; \bar{s}[\sigma] \rangle\} \rrbracket \\ &= \text{CoData}([\llbracket \overline{O}[\sigma] \rrbracket], [\llbracket \langle V[\sigma]; \bar{s}[\sigma] \rangle \rrbracket], i) \end{aligned}$$

By IH on each sub-expression of $\overline{O}$, V , and $\bar{s}$:

$$= \text{CoData}([\llbracket \overline{O} \rrbracket][\llbracket \sigma \rrbracket], [\llbracket \langle V; \bar{s} \rangle \rrbracket][\llbracket \sigma \rrbracket], i) = \llbracket \{\overline{O} \approx_i \langle V; \bar{s} \rangle\} \rrbracket[\llbracket \sigma \rrbracket].$$

- $(e = (q = e'), \text{observation})$ An observation $q = e'$ is not an expression in context; its body e' is translated as $\lambda y. \text{let } x_i := y[x_i] \text{ in } \llbracket e' \rrbracket$. Substitution acts on e' (avoiding the copattern-bound variables $BV(q)$):

$$\llbracket (q = e')[\sigma] \rrbracket = \llbracket q = e'[\sigma_{-BV(q)}] \rrbracket$$

$$= \text{Obs}(\llbracket q \rrbracket, \lambda y. \overline{\text{let } x_i := y[x_i] \text{ in } \llbracket e'[\sigma_{-BV(q)}] \rrbracket})$$

By IH on $e' := \text{Obs}(\llbracket q \rrbracket, \lambda y. \overline{\text{let } x_i := y[x_i] \text{ in } \llbracket e' \rrbracket[\llbracket \sigma_{-BV(q)} \rrbracket]}) = \llbracket (q = e') \rrbracket[\llbracket \sigma \rrbracket]$.

□

LEMMA B.11 (SIM-FULL-MATCH). *If $M \xrightarrow{\text{full-match}} N$ for any Fw_{py}^{co} terms, then there exists a Fw_{py} term T such that $\llbracket M \rrbracket \rightarrow^* T$ and $T = \llbracket N \rrbracket$ (modulo observational equivalence).*

PROOF. Since s is an $SStep$, it is either an attribute access ($\square.a$) or a call ($\square(V)$). This action will be performed on the translation of a $\{_ \approx_i _ \}$, which is an object of class CoData . In the case of an attribute access, the translation of M don't have any attributes besides obvs , i , st , so the only possible outcome is an else hit.

$$\text{CoData}(\text{self.obvs}, \text{self.st} ++ [\text{TAttr}(a)], \text{self.i}.comatch(\{\}))$$

The else implementation of the *CoData* class starts a new *comatch* absorbing the current action. In the case of a call, the translation of M has a **call** member which does a similar action:

$$\text{CoData}(\mathbf{self}.obvs, \mathbf{self}.st \mathbin{++} [\text{TApp}(\text{args})], \mathbf{self}.i).\text{comatch}(\{\})$$

By hypothesis, $q \approx \langle V ; \bar{s}, s \rangle = \mathbf{match} \sigma$ for the i -th observation. By Lemma 5.3, the *Obs.comatch* call result is $\{x : \llbracket V \rrbracket\}$. Since this is not **None**, *CoData.comatch* calls $o.out(\mathbf{self}, r)$. The attribute $o.out$ holds the multi-argument lambda $\lambda(\mathbf{self}, y). \overline{\text{let } x_i := y[\cdot x_i] \text{ in } \llbracket e \rrbracket}$ (from the translation of $(q = e)$ in fig. 15). The call $o.out(\mathbf{self}, r)$ therefore β -reduces by binding the co-object to $\mathbf{self}$ and the binding dictionary $r = \{x : \llbracket V \rrbracket\}$ to y , then evaluating each $\text{let } x_i := y[\cdot x_i] \text{ in}$ to bind the copattern variables. The result is $\llbracket e \rrbracket[\text{CoData}(\dots)/\mathbf{self}, \llbracket V_i \rrbracket/x_i]$. By Lemma B.10, this equals $\llbracket e[\llbracket \bar{O} \approx_0 \langle V ; \rangle \rrbracket / \mathbf{self}, \sigma] \rrbracket = \llbracket N \rrbracket$, closing the simulation. $\square$

LEMMA B.12 (SIM-PART-MATCH). *If $M \xrightarrow{\text{part-match}} N$ for any $\text{Fw}_{p_y}^{\text{co}}$ terms, then there exists a Fw_{p_y} term T such that $\llbracket M \rrbracket \rightarrow^* T$ and $T = \llbracket N \rrbracket$ (modulo observational equivalence).*

PROOF. As in the proof of Lemma B.11, the *else/call* handler creates a new *CoData* object with the extended setting and calls *comatch*. By hypothesis, $q \approx \langle V ; \bar{s}, s \rangle = \mathbf{partial} \sigma$ for the i -th observation. By Lemma 5.3, the *Obs.comatch* call result is $\{x : \llbracket V \rrbracket\}$. Since this is a partial match ($\text{len}(o.q) > \text{len}(\mathbf{self}.st)$), *CoData.comatch* returns $\mathbf{self}$, which is the *CoData* object with setting $\llbracket \langle V ; \bar{s}, s \rangle \rrbracket$. This is exactly $\llbracket \llbracket \bar{O} \approx_i \langle V ; \bar{s}, s \rangle \rrbracket \rrbracket = \llbracket N \rrbracket$, closing the simulation. $\square$

LEMMA B.13 (SIM-FAIL-MATCH). *If $M \xrightarrow{\text{fail-match}} N$ for any $\text{Fw}_{p_y}^{\text{co}}$ terms, then there exists a Fw_{p_y} term T such that $\llbracket M \rrbracket \rightarrow^* T$ and $T = \llbracket N \rrbracket$ (modulo observational equivalence).*

PROOF. As in the proof of Lemma B.11, the *else/call* handler creates a new *CoData* object bb_s with the extended setting $\llbracket \langle V ; \bar{s}, s \rangle \rrbracket$ and calls *comatch*. By hypothesis, $q \approx \langle V ; \bar{s}, s \rangle = \mathbf{fail}$ for the i -th observation. By Lemma 5.3, the *Obs.comatch* call result is **None**.

Since this is a failure, *CoData.comatch* constructs

$$bb' = \text{CoData}(\mathbf{self}.obvs, [\mathbf{self}.st[0]], \mathbf{self}.i+1)$$

which is the fresh co-object $\llbracket \bar{O} \approx_{i+1} \langle V ; \rangle \rrbracket$ translated and allocated. It then executes

$$\text{fold}(\lambda a, f.f.\text{unwind}(a), \text{drop}(1, \mathbf{self}.st), bb')$$

where $\text{drop}(1, \mathbf{self}.st) = [\llbracket s_1 \rrbracket, \dots, \llbracket s_k \rrbracket, \llbracket s \rrbracket]$ are the accumulated interaction steps. Note that $\text{drop}(1, xs)$ is definable in Fw_{p_y} as a *fold* that accumulates entries whose key is greater than 0, so no additional primitives are required.

We connect the fold result to $\llbracket N \rrbracket$. By the definitions of *TAttr* and *TApp*:

$$\begin{aligned} \text{TAttr}(a).\text{unwind}(e) &= e.a && \text{(attribute access, corresponding to SStep } \square.a) \\ \text{TApp}(\text{args}).\text{unwind}(e) &= e(\text{args}) && \text{(call, corresponding to SStep } \square(V)) \end{aligned}$$

The fold therefore applies each step in $\bar{s}, s$ in order to bb' , building up:

$$bb' \xrightarrow{\llbracket s_1 \rrbracket} s_1[bb'] \xrightarrow{\llbracket s_2 \rrbracket} \dots \xrightarrow{\llbracket s_k \rrbracket} s_k[\dots s_1[bb'] \dots] \xrightarrow{\llbracket s \rrbracket} T$$

This mirrors exactly the *fwcop* target $N = s[\llbracket \square ; \bar{s} \rrbracket[\llbracket \bar{O} \approx_{i+1} \langle V ; \rangle \rrbracket]]$, which applies the same steps to the same fresh co-object. Since $bb' = \llbracket \llbracket \bar{O} \approx_{i+1} \langle V ; \rangle \rrbracket \rrbracket \rightarrow^1 bb'_{\text{alloc}}$ (one allocation step), and the fold over $\llbracket \bar{s}, s \rrbracket$ corresponds to $\llbracket \llbracket \square ; \bar{s}, s \rrbracket[\cdot] \rrbracket$ by Lemma B.7, we obtain $T = \llbracket N \rrbracket$ modulo observational equivalence, closing the simulation. $\square$

THEOREM 5.1 ($\llbracket _ \rrbracket$ FORWARDS SIMULATION). *If $M \rightarrow N$ in $Fw_{p_y}^{c_o}$, then there exists a Fw_{p_y} term T such that $\llbracket M \rrbracket \rightarrow^* T$ and $T = \llbracket N \rrbracket$ (modulo observational equivalence).*

PROOF. By case analysis on the reduction step. For the inherited Fw_{p_y} reduction rules, $\llbracket M \rrbracket = M$ and $\llbracket N \rrbracket = N$ by Lemma B.8, so the same step $M \rightarrow N$ applies directly, giving $T = \llbracket N \rrbracket$ with $\llbracket M \rrbracket \rightarrow^1 T$. The cases for the new $Fw_{p_y}^{c_o}$ constructs are covered by Lemma B.9, Lemma B.11, Lemma B.12, and Lemma B.13, each of which produces a T with $\llbracket M \rrbracket \rightarrow^* T$ and $T = \llbracket N \rrbracket$ modulo observational equivalence. $\square$